ZURFUR · RELEASE 0.1

Technical Design Document

Release 0.1, `pre-alpha-v0.1.0`: “Sign in with Bluesky and browse My Den”

Release	0.1	**Document version**	4
Date	2026-10-10	**Status**	In review
Prepared by	Kestrel, the tech lead's release planner	**Approver**	The Engineer (Zuri)

Workflow Draft → In review → Signed

In one page

One path, three features, and one side feature. Browser, then the SvelteKit web app, then the axum API, then `application`, then Atta (the Den engine) and the adapters. Sign-in runs down the left of that path, My Den down the middle, and the Explorer frame sits on top of both (“Architecture overview”). 0.1 joins sign-in and My Den, without character storage (the Product Design Document’s D-1, your ruling). Version 3 adds your two other rulings of 2026-10-09: time limits on every call from the web server to the API, with the gate’s own “isn’t answering” page (Feature 2, D-35), and bug reports, a side feature that moves to 0.2 if it isn’t on `main` by unit 9 (Feature 4, D-34). Version 4 records your rulings of 2026-10-09 on every decision here.

Where each feature stands.

- **Sign-in** is built: seven slices, QA’d and security-reviewed, with a final verdict of “ship” (Heron, 2026-10-08). Six PRs are open: #238 and #239 into the branch, and #240 to #243 stacked above slice 3, the resolver (they are slices 4, 5, 7 and 6). Slice 3’s own PR opens once #238 and #239 merge. What remains: merges, five small follow-up slices (four before `main`: the identity-only scope, and the session check’s bounded wait for your profile among them) and two corpus updates.
- **The Explorer frame** and My Den on mock data are built and reviewed: #245 into `feature/web-explorer-shell`, and #246 to #249 stacked above it; slice 2 is still a draft. The shell goes to `main` before the identity branch; the time limits follow it straight to `main`.
- **My Den** is designed: the diagrams were approved on 2026-10-07, the rulings are in the corpus (decision page 2026100701), and den.proto is drafted, QA’d, confirmed by the den lead and approved by you as a whole (revision 2.1, D-6). Nothing is built. On `main`, Atta is a six-line stub; your 83-line draft, with a `Key(Uuid)` that the rulings replace, sits uncommitted in the primary checkout, and Atta’s read side builds on it (your ruling).
- **Bug reports** are proposed and QA’d (`proposals/bug-reports.md`, 19 challenges closed); nothing is built.

Data. My Den adds no table: it projects nodes from today’s rows and serves metadata, plus the soft-deleted word (D-16), from PostgreSQL on the everyday stack and from the in-memory fixture world on the dev profile. Bug reports add one private table, `bug_report` (D-37). The identity branch adds two migrations; commission adds one index. In pre-alpha none of these is a one-way door (D-36, your ruling); the one-time delete of your stored sign-in sessions still is.

Security. Identity hardening is reviewed. My Den’s path handling was reviewed before any code (2026-10-07). The reads that bypass the Den move behind den’s access check before the end-of-slice review starts. The time limits close Heron’s note that the gate’s backend calls have no bound, and never read a timeout as “signed out”. Bug reports get their own review, by Mantis rather than Heron. Each security piece gets its own review, and My Den ends with a full one.

Decisions here: 13, all ruled by you on 2026-10-09, each as Claude answered it, except that D-36 adds your own exception. The one door, den.proto, carrying six (D-6 to D-10, D-14), is approved as a whole; its text still comes to you on its PR (WP-13). Two smaller doors you review

> on their PRs: the one-time session delete (D-32) and `bug_report.proto` (your ruling on carry-over row 84). You approved D-15, with its inner question answered and the link question deferred, and the joined component diagram (Feature 3). The carry-over's three technical questions are settled: Atta's read side builds on your draft, no code path serves a payload, and its first slices no longer wait on D-15.

Point, don't copy. This document summarizes and points. The detail is in: `.understand/20261008-hardened-resolver-design.md` (sign-in); `.understand/20261008-web-explorer-shell.md` (the frame) and `objectives/web.md`, "Planned ahead: loading states" (the time limits); `proposals/bug-reports.md` (bug reports); `.understand/20261006-my-den-architecture.md`, `.understand/20261006-my-den-question-sheet.md` and `.understand/20261007-den-proto-draft.md` (My Den); and the corpus pages 2026093001 (The Den) and 2026100701 (My Den rulings). Where a ruling and this summary differ, the ruling wins.

1 Architecture overview

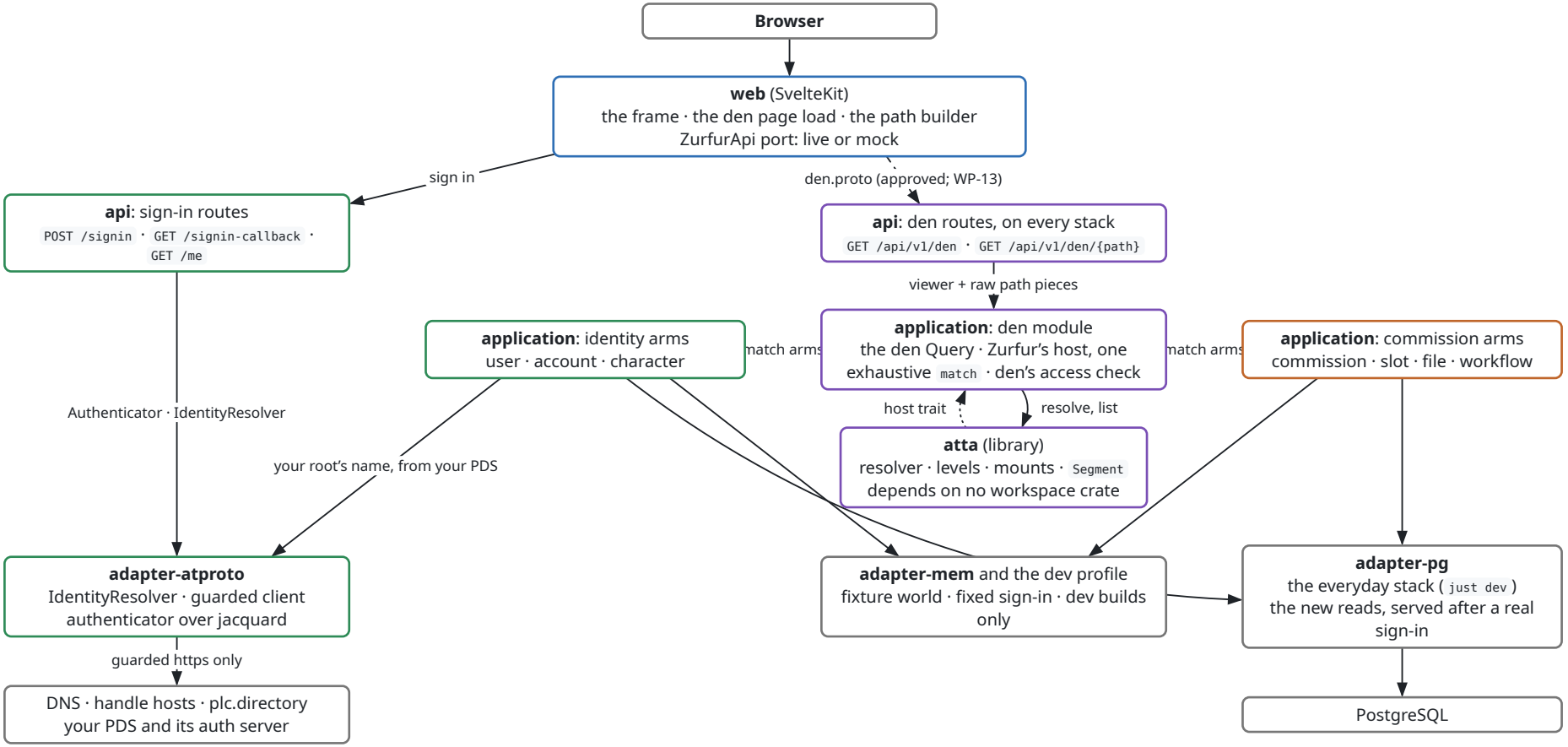


Figure 1: How 0.1 fits together. Box outlines show the owning team. Solid arrows are calls; the dotted arrow is Atta calling back into its host; the dashed line is den.proto, approved by you (D-6) and not yet in the contract (WP-13). Each arm reads through ports: PostgreSQL on the everyday stack, the in-memory fixture world on the dev profile.

web team (Fennec) · den team (Badger) · identity team (Fox) · commission team (Wolf) · shared ground (tech lead)

The dependency rule (Domains and Applications, 11763713; My Den rulings, decision 1): adapters depend on `domain`, never the reverse; `composition` alone picks the live adapters; `application` depends on `atta`; `atta` depends on no workspace crate, and a guard test keeps it so; the API never imports `atta`, and hands raw path pieces to the den Query. Ports, `shared`, `composition` and `domain` are left out of the picture to keep it readable.

Not drawn, to keep it readable: the time limits live inside the web box, in its one fetch helper and its sign-in gate (Feature 2); bug reports run beside the path, as a form action, one route in the API's cookie surface, one table and an operator command (Feature 4).

2 Feature 1: sign in with Bluesky, hardened

2.1 Architecture

One resolver does every identity lookup: handle to DID, DID to handle, and the DID document behind them (`.understand/20261008-hardened-resolver-design.md`, §1 to §3).

- **The port:** `IdentityResolver`, in `domain`, the only door for code outside `adapter-atproto`. Its error, `ResolveError`, has four variants (not found, not confirmed, refused, unavailable), and anonymous routes collapse them to one answer.
- **The adapter:** `AtprotoIdentityResolver`, with its own DNS client (hickory 0.26.3) and one guarded HTTP client, `GuardedHttp`. Every URL is a `PublicHttpsUrl`, a validated type, so a check can't be skipped.
- **OAuth** still runs on jacquard, but through `JacquardBridge`, which sends every request through `GuardedHttp` and every DID document through the resolver. Sign-in `start` takes an `AtHandle`, binds the DID before the PAR, and checks the authorization endpoint first.
- **Guards:** a test fails the build if production code names a known door around the client (bare reqwest, jacquard's convenience clients, `JacquardResolver`, Bluesky's AppView host). A log filter drops every hickory event, because hickory logs the queried name.
- The names `IdentityResolver`, `AtHandle` and `ResolveError` were approved by you on 2026-10-08.

2.2 Contracts

Surface	What it promises
`POST /signin`	Takes a handle, trimmed at the route and then parsed strictly by the atproto handle spec. Anything else, and every lookup failure, gets one `422` with a byte-identical body. Success is a `303` to the authorization endpoint, with the browser-binding cookie, which the SvelteKit login action relays.
`GET /signin-callback`	Routed straight to axum (a design rule of the one origin, 11763713). Failures redirect to `/login?error=<code>`, with the codes `denied`, `invalid_callback`, `exchange_failed` and `account_mismatch`, plus `browser_mismatch` under D-5. Every answer clears the binding cookie today; D-12 changes that to "only on a match".
Cookies	`zurfur.sid`, the session (unchanged). The binding cookie is `__Host-zurfur.signin` in staging and production, and `zurfur.signin` in dev: `HttpOnly`, `SameSite=Lax`, `Path=/`, 10 minutes. The server stores only its SHA-256.
`GET /me`	The profile now comes through the resolver: the handle from the both-ways check, the PDS from the validated document. An outage errors instead of caching a stripped profile.

2.3 Data

- **Two migrations on the identity branch:** `20261009002316_auth_request_browser_binding.sql` (a nullable hash column; older rows fail closed) and `20261009002000_client_session_by_session_id.sql` (an index, so the cleanup that deletes by session id doesn't scan the whole table).
- **One check before the merge:** the second version's time part is exactly 00:20:00. CLAUDE.md warns that hand-typed, round versions collide across branches. Confirm it came from `just migrate-add` (WP-1 in the Delivery Plan).
- No resolution cache. The limits are constants an operator can't loosen: 10 s and 512 KiB per fetch, 5 s per DNS lookup, 20 s per port call, 30 s for sign-in `start` and for the callback.
- **The old sessions go once:** the scope slice deletes the stored sign-in sessions, which carry the old full-write grant, and nothing else; not `just db-reset`, which would also wipe your real Accounts (Product Design Document, D-32). **Claude's implementation choice, which you can overturn:** a migration made with `just migrate-add` deletes every stored OAuth session and pending sign-in (`client_session`, `auth_request`) and nothing else. Only a migration keeps them out: the API runs every migration at boot, and sqlx refuses to start a build that lacks a migration the database has run, so no older build can store a full-write session there again. The same rule means a database that has run the identity branch can't run `main` until the branch lands, so the real-sign-in check (M0) runs on a workspace's own database (the Delivery Plan).

2.4 Security

Check	What it stops	Status
Handles only at sign-in	A typed `https://…` URL choosing the server Zurfur talks to	Built, reviewed
Private addresses refused after DNS; IP-literal hosts refused	A handle or document sending Zurfur to `10.x`, `127.x` or the cloud metadata service	Built, reviewed
https on port 443 only; certificates checked; no redirects; no proxy	Downgrades, port probing, "302 to an internal host", a proxy variable rerouting traffic	Built, reviewed
Body cap and timeouts	A hostile host exhausting memory or holding a task	Built, reviewed
No third-party fallback	Bluesky learning every handle typed into Zurfur, or answering in place of DNS	Built, reviewed
Handle and DID checked both ways	A domain claiming someone else's account	Built, reviewed
Authorization endpoint passes the URL policy before the PAR	Sending a browser to a plain-http or internal URL (the alpha's "https-only PDS redirect")	Built, reviewed
DID bound at start, checked at the callback	Finishing as a different account from the handle typed; the tokens are revoked	Built, reviewed
Sign-in bound to the browser	Login CSRF: a victim opening an attacker's callback link	Built, reviewed
Handles out of logs	A log tying a visitor to Zurfur	Built, reviewed
PDS address must be a bare origin	A non-conforming address reaching callers that treat the PDS as an origin (no security effect)	Ruled (D-11); WP-3 builds it
Sign-in asks only for `atproto`	A stored session asking for write access to your real repo, which nothing in 0.1 uses (Product Design Document, D-17). That an `atproto`-only token can't write is unverified: neither atproto spec says what it grants.	Ruled; the scope slice (WP-26) builds it

Residuals, stated so nobody assumes otherwise (the design's sign-in and review-plan sections; the security review's final section): Zurfur's own public addresses pass the filter on port 443, so Zurfur must never trust its own public IP; only the well-known NAT64 prefix is unwrapped; there is no network-layer egress guard yet; jacquard saves a session before the account check runs, and a mismatch then revokes and deletes it; whoever knows a victim's pending `state` can cancel that sign-in (low, accepted). The deployment checklist (an egress IP no internal service trusts, IMDSv2 only, cookie redaction in production logs, no user content on any `zurfur.app` subdomain, the ingress carve-outs proved with `scripts/web-smoke.sh`) waits for the release that deploys.

2.5 Sequence: signing in

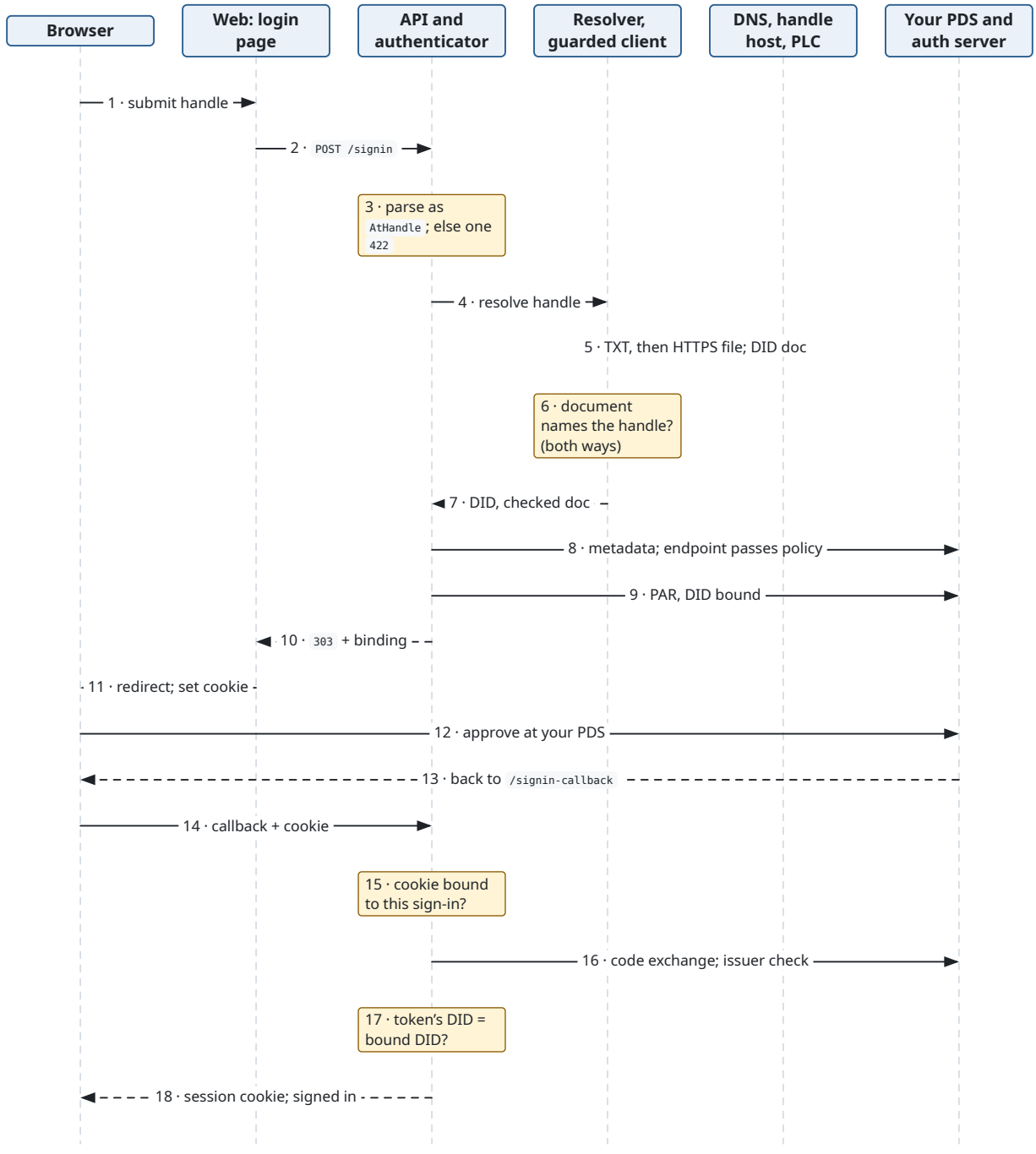


Figure 2: Signing in with a handle. Notes (shaded) are checks inside one component; dashed arrows are replies.

What fails where. Step 3 or any lookup in steps 4 to 9: the one `422`. Step 15: refused before any code exchange (`browser_mismatch` under D-5). Step 17: the tokens are revoked and deleted, and the visitor lands on `/login?error=account_mismatch`. Step 16's issuer check fetches the DID document through the resolver, so no document reaches jacquard unchecked. Full detail: the design's diagram (b) and its check table.

3 Feature 2: the Explorer frame and the neutral dark style

3.1 Architecture

From `.understand/20261008-web-explorer-shell.md` (diagrams A to C, sections 2, 3 and 10):

- **Two route groups,** `(public)` and `(session)`; no URL changes. `(session)/+layout.svelte` is the frame, called AppShell: TopBar, Sidebar (Tree and ViewOptions) and the pane. It holds the tree's memory and **reads nothing from the Den.**
- **One route serves the root and everything under it:** `den/[...path]/+page.server.ts`, whose load is the only code that calls the Den. It calls `GET /api/v1/den` for an empty path and `GET /api/v1/den/{path}` otherwise.
- **Components:** the core ones carry the design language's names (Button, IconButton, Toggle, TreeItem, PathBar, Badge, VisibilityMarker, Avatar, Dialog, EmptyState, Skeleton), in `lib/components/ui/`; TextInput, Checkbox and InlineMessage arrive with bug reports (Feature 4). The frame's own compositions (AppShell, TopBar, Sidebar, Tree, ViewOptions, NodeHeader, EntryList, ProblemNote) are in `lib/components/explorer/`.
- **One frame on every stack.** The shell's slice 1b hides the tree where no Den is served, so `main`'s `just dev` shows no Den between the shell's merge and My Den's. A package of its own, My Den on every stack (WP-40), deletes that switch and its no-Den frame right after the wiring (D-1; the tech lead's call).
- **Style:** plain CSS and Svelte's scoped styles, no new dependency. Tokens in three tiers in `lib/styles/tokens.css` (palette, semantic, layout), element defaults in `base.css`; dark values on `:root`, light values defined and unused; sizes in rem; fonts and icons self-hosted.

3.2 Contracts

- The web reads My Den through the `ZurfurApi` port's `denRoot` and `denNode`, in a live and a mock version.
- Both produce the web's own plain Den view (`lib/api/den.ts`: branded strings, unions on access, no `null`). That is why the frame and the mock Den don't wait for den.proto.
- **The tolerant reader:** an access the web doesn't know shows as a card; an unknown kind gets a generic marker; an open node with unknown content shows "content not shown yet"; an answer without a node is a contract error, shown as the frame's error view.

3.3 Data

Nothing is persisted. The tree's memory lives in the frame's Svelte context, never in a module-level store, and is keyed by the session's DID; a mismatch clears it, and a change of user is always a full page load. It is cleared when "Show archived and deactivated" flips and after a successful form action. The toggle's flag rides only in the address, and the server sends each page's link with the flag both on and off, so the browser never builds a Den address itself.

3.4 Security

- **One path builder,** used only in the den page's server load: it refuses empty, `.` and `..` pieces and more than 16 pieces, encodes each piece, checks that the parsed pathname equals the built string, takes query parameters only from an allowlist, and builds every Den link. It also hardens today's account-delete call (path-handling review, question sheet §12).
- `handleError` logs only the route id; the error view never shows an error's message; `Referrer-Policy: same-origin` on signed-in pages. Already on `main`: `Cache-Control: private, no-store` on signed-in pages (PR #233), and the API proxy forwarding only `zurfur.sid` (PR #234).
- Names are text, never HTML (no `{@html}`), isolated with `<bdi>`. Tree links preload on tap, not on hover. Page titles are generic ("My Den · Zurfur").
- **A cap on parallel Den reads:** a deep page load reads up to 17 folders. The wiring caps a page at 4 reads in flight, which should also guard PostgreSQL's pool of 5 connections with a 5 s wait (`adapter-pg/src/lib.rs`); that is unverified, and the `just dev` walk opens a deep address to check it. A Den read holds a connection per query, not for its whole run.

3.5 Time limits and the "isn't answering" page

New in version 3, by your ruling on D-35 (the Product Design Document), from the web lead's design after Crow's QA (`objectives/web.md`, "Planned ahead: loading states", the time-limits bullets). It closes

Heron's note that the gate's own `/me` call, and the other backend calls, have no bound ("Copilot round 2", row 8, in `.understand/20261008-explorer-shell-security-steps-1b-2.md`). One small web package (WP-41), straight to `main` after the shell, and one small identity package on the identity branch (WP-49).

How it works (on the shell's branch today, every call to the API goes through one helper, `backendFetch` in `lib/server/api/zurfur-api.ts`, and the gate is `sessionGate` in `lib/server/session-gate.ts`):

- **The limits, in one place.** `backendFetch` gives every call an abort signal: 5 s for the session check (`/me`), 10 s for every other call; sign-out keeps its own 5 s (`SIGNOUT_TIMEOUT_MS`). A call added later, the den reads included, is bounded with no code of its own. *Claude's implementation choice, which the web lead can overturn.*
- **The whole request: 15 s,** a deadline set once per request in the server hooks; each call's signal is the earlier of its own limit and that deadline. So a page whose reads hang (up to 17 Den reads, 4 at a time) can't hold the browser for 50 s.
- **A call that runs out fails as `NetworkFailure`,** the error an unreachable backend already gives.
- **The gate stops reading that as "signed out".** Today it asks with `sessionOrAnonymous`, which turns a network failure into "anonymous", and redirects to `/login`. It asks with `optionalSession` instead: a `401` still redirects to `/login`; a network failure, a timeout included, gets the gate's own page.
- **The gate's page.** Fixed HTML, returned by the gate itself: never thrown, which would reach SvelteKit's bare fallback page without any of the signed-in headers (Crow's QA note), and never the frame, where a form action could run with no session behind it (Heron's round 2, row 9: actions run before layout loads). It reads "Zurfur isn't answering right now." with "Try again", which reloads the same address without writing the request's address into the page; if any part of a request is ever written into it, it is HTML-escaped first, and a test requests a path carrying `'`, `"`, `<` and `>`. This page is the one place the web writes HTML outside Svelte, and the only script policy today is `frame-ancestors 'none'`. Status `503` (proposed, for the web lead to confirm). Because the gate returns it through the hook chain, the outer hooks stamp it like any signed-in response: `Cache-Control: private, no-store`, `Referrer-Policy: same-origin`, `frame-ancestors 'none'` and `X-Frame-Options: DENY`. The session cookie is untouched.
- **What the browser does with that page,** in the installed SvelteKit (2.70.1; Owl's QA read `src/runtime/app/forms.js` and `src/runtime/client/client.js`, and Kestrel re-read `forms.js`). A full page load shows it. A navigation made by the page's script asks for `__data.json`; the non-JSON answer fails, SvelteKit asks again for the root layout's data (this app's root layout has a server load), then falls back to a full page load, which shows the page: against a hung backend that is up to three 5 s session checks. An enhanced form post that meets the page fails to parse as JSON and becomes an error result, which by default renders the frame's error view with status 500. No action ran, because the gate answered first, so this is a display gap, not a leak. **WP-41 decides** how a form post meets the gate: an answer the client reads as a failure that says Zurfur isn't answering, or the frame's error view with those words, never "signed out" and never a status number alone. The bug-report form's own handler keeps its text and says it couldn't send (WP-45).
- **Elsewhere a timeout behaves as an unreachable backend does today.** An open item's Den read fails to the frame's error view, never "Nothing here" (`den.ts`, `readOutcome`). A folder read above the item is dropped, so the tree opens less far (`readAncestors`); the "Couldn't load. Retry." row waits with the loading states. The root layout on a public page still renders signed-out: it holds nothing private. A form whose call runs out fails as it does when the backend is down; the bug-report form keeps its text (Feature 4). A write that runs out at the web may still have gone through at the API: a duplicate bug report is harmless (it counts toward the hourly 10), but founding an Account could report a failure for an Account that was founded, and a retry is then refused because the handle is taken. WP-41 words a timed-out write as "it may have gone through" where that matters (the web lead's wording).
- **Starting a sign-in** is a call like any other: the web stops waiting at 10 s, inside the API's own 30 s (Feature 1, "Data"). The login form then says "Zurfur isn't answering right now. Please try again." instead of the error page (the tech lead's call, 2026-10-09; in WP-41).

The session check's wait for your profile (WP-49, identity). On the identity branch `GET /me` loads your user, then your profile read-through: the profile cache (with a freshness window), else your PDS,

through the resolver, whose limits are 10 s per fetch and 20 s per port call (`application/src/user.rs`, `resolve_profile`; `adapter-atproto/src/resolver/limits.rs`). On a cold cache and a slow PDS, `/me` could outrun the 5 s session check, and the gate would say Zurfur isn't answering while it is. So `/me` waits only a short, fixed time for the profile, inside the 5 s, and otherwise answers with the bare DID, which its contract already allows ("an unresolved profile omits the keys"); the top bar then shows your DID (the tech lead's call, 2026-10-09). Identity names the wait's length in the package; your real root (WP-32) can reuse the same bounded lookup. It still matters whether the profile fetch carries on after `/me` has answered: today's read-through writes the cache only when the fetch returns, so if the bound drops the fetch, a PDS always slower than the bound leaves your DID in the top bar on every page. So WP-49 lets the fetch finish in the background, one lookup per DID (WP-32's rule), and a test shows a slow PDS's profile on a later load.

Tests (WP-41 and WP-49): a backend that never answers `/me` gets the gate's page within about 5 s, with the four headers and the cookie kept; a form action behind the gate records that it never ran, and an enhanced post shows the agreed failure; a script navigation ends on the page; "Try again" with a path carrying `'`, `"`, `<` and `>` is escaped; a `401` still redirects to `/login`; a page whose reads all hang ends within 15 s; sign-out still runs without the gate, bounded at 5 s; a sign-in start that runs out shows the login form's message; a `/me` whose profile source never answers returns the bare DID within its wait, and the profile shows on a later load. The Den reads' cases (an open item's read that runs out shows the error view; a folder above it is dropped) are tested in the wiring (WP-10), which brings the Den reads, by failing a read the way a timeout does. **Review:** Heron (session handling; his note), with Crow's QA, and Magpie's for WP-49.

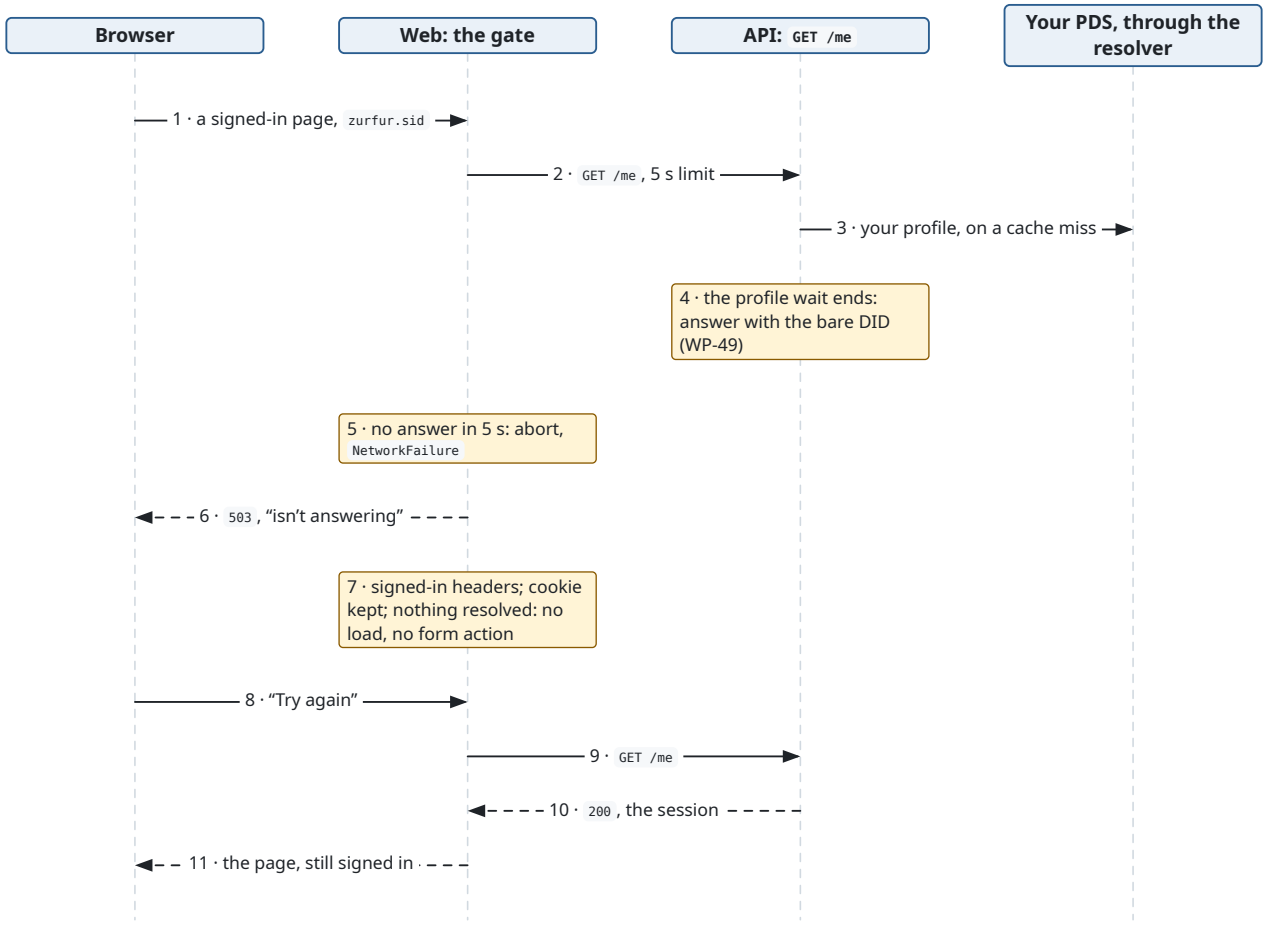


Figure 3: A signed-in page when the backend doesn't answer (D-35). Shaded notes are checks inside one component; dashed arrows are replies.

Step 4 is why step 5 shouldn't happen while the API is up: with WP-49, `/me` answers within its own wait whatever your PDS does. Step 5 is what a stopped or hung API gives.

4 Feature 3: My Den, read-only

4.1 Architecture

From the approved diagrams 1 and 2 (`.understand/20261006-my-den-architecture.md`) and the My Den rulings, decisions 1 to 3:

- **Atta is a library below `application`.** It knows nodes, folders, files, links, mounts, the three levels and the level formula, and nothing of Zurfur.
- **One host trait** is how Atta reaches Zurfur's types: object-safe, used through `&dyn`, async through `async-trait`, with `Send` futures, speaking only Atta's own value types (`Segment`, `NodeName`, `Level`, `Metadata`, an opaque type key, and, under D-15, the reach, `Listing` or `Path`, and `Admission`, `Admit` or `Refuse`).
- **One implementation,** in `application`'s den module: an exhaustive `match` over Zurfur's type enum. Each arm calls the owning team's module, with no runtime registry.
- **The den Query,** a `#[use_case]`, turns the raw path pieces from the API into `Segment`s and calls Atta. The API never imports `atta`.
- **A key is its path segment,** so a node's identity is its real path (My Den rulings, decision 2). Atta's read side builds on your uncommitted draft in the primary checkout (your ruling on carry-over row 9), and the draft's `Key(Uuid)` goes, with the other changes the rulings force: `parent` and `symlink_target` become real paths, an opaque type key replaces the empty `Type` enum, and `NodeName` and the segment get hand-written `TryFrom` and `FromStr`. The payload hash, `Payload` and attributes wait for the first payload.
- **Standing covers the real subtree** and never crosses a mount (decision 3). The recorded fallback is that each node answers alone.
- **Two stacks, one code path** (D-1). The everyday stack (`just dev`) serves My Den from PostgreSQL after a real sign-in. The dev profile serves the in-memory fixture world: `adapter-mem` behind a Cargo feature, refusing to start unless the environment is `dev`, signing in as a fixture user. The arms reach storage only through ports, so one codebase serves both. The dev profile's owner is the tech lead, and a builder builds it; the web builds its "sign in as" page.
- **Den's access check.** One check in the den module for the reads that bypass the Den: "may this viewer open the node at this real path?" It runs Atta's own resolve, not a second copy of the rule, and answers "opens" or the one not-found. A card doesn't count as opening, and real paths stay on the server. Each read asks about the node it serves: the Accounts list about each Account, `/me` about your own root, the changelog read about the commission's `changelog` node, a download about the file's own node. Wire shapes don't change.
- **Your real root.** On the everyday stack the user arm names your root from your own PDS, through the hardened resolver: the display name, else a handle that checks out both ways, else your DID. It reads the profile cache first, keeps one lookup per DID in flight, and falls back to your DID after a short wait, so an outage never reaches Atta as an error. A page makes up to 17 Den reads, so while a lookup for your DID is running or has just failed, later reads fall back at once (identity's design to settle in WP-32). The cache changes which statements a read runs, so the equal-statements tests fix the cache's state first. Zurfur resolves only your own DID, from the session. The profile ports are already among the application's ports (`application/src/app.rs`), so the den Query stays blind to profiles.
- **Ownership:** den owns `atta`, `application/src/den*`, `api/src/routes/den*` and den's access check; identity and commission own their arms and read ports, and rewire their own reads onto the check (identity: the Accounts list and `/me`; commission: the changelog read and the file download); the contract and the adapters are shared ground.
- **Type names are provisional** (`commission`, `commission.attachments` and so on; ruling P10). Either the type-catalog decision lands before the alpha, or these names ship as they are.

4.2 Contract: den.proto

The full draft, with every field's ruling, is `.understand/20261007-den-proto-draft.md`. In short:

Piece	What it says
Routes	`GET /api/v1/den` reads the viewer's own root; `GET /api/v1/den/{path=**}` reads below it. Both are served on every stack (D-1). The session comes first (`401`), then the query is checked, then the path. The viewer comes from the session alone: no request field names a viewer or a root. Only Den paths travel, never a real path.
Request	`path` : each piece percent-encoded once, joined by `/` ; at most 16 pieces of 1 to 2048 bytes of `[A-Za-z0-9._:%~-]` , never `.` or `..` . `includeDeleted` and `pageToken` as query parameters (spelled as you ruled, D-7).
`DenNode`	`segments` (the Den path; empty on a card), `name` (from what the node holds, never its key), `type` (provisional), `kind` (directory, file or symlink; unset on a card), `mount` , and `oneof access { open | card }` , where unset or unknown renders as a card.
`DenOpen`	Only for viewers who can open the node: `own_level` (private, listed, public; an unknown value shows no level), `content_not_shown` (fixed per type, never read from the data) and `soft_delete` , a word naming the soft-deleted state, today `archived` for a commission or `deactivated` for an Account, absent when live (D-4; its name is D-14). A word the client doesn't know still shows the node as soft-deleted, never as live.
Content	`oneof content { listing | file | link }` . The node is required, so a stray body is a contract error; content is optional, so a new kind of content reaches an older client, the command-line tool included, as "content not shown yet". A path to a file answers `200` with an empty `file` .
`DenListing`	`entries` in the server's order (clients keep it within each kind) and `next_page_token` : opaque, a version byte and an offset into the viewer's filtered listing, with no names, hashes or counts. The page is cut after filtering.
Errors	Every miss is one `404` : `urn:zurfur:error:node-not-found` , code `node_not_found` . An unknown query parameter is one `400` : `urn:zurfur:error:bad-request` , code `unknown_parameter` , a new registry entry. A bad value or a malformed or stale token is one `422` , `invalid_request` .
Size and checks	11 messages, 26 fields, 2 rpcs. `buf lint` (the repo's `STANDARD` config) and `buf build` pass in a scratch copy. The contract stays at v1 until the alpha (rulings P1 and C13), and under your ruling on the Product Design Document's D-19 the pre-alpha tag doesn't close it. You approved revision 2.1 as a whole (D-6); its text comes to you on WP-13's PR, with D-19's line in the lifecycle file. The route-test pin moves in the same PR as the proto, and lists the two den routes like any other, with no dev-profile marking.

Elsewhere in the contract. Under the Product Design Document's D-28, the owner's commission list (`GET /commissions`) and its contract entry go, as their own declared baseline-reset PR; the create response keeps its shape. The baseline-reset line has to be in `contract/VERSIONING.md` first (`v2-carryover.md` , row 14).

4.3 Data

- **No new table.** Nodes are projected from today's rows, and the lasting storage shape is settled with the first payload (your ruling, question P14).
- **Metadata only, plus the soft-deleted word:** the word is projected from today's rows and is unverified, like the name and level (D-16, your ruling). **No code path serves a payload:** the host trait has no payload method, so the payload hash and MAC check (The Den, decision 25) is recorded as not exercised (your approval of carry-over row 12, 2026-10-09, which replaces the verifier that refused every payload under your ruling to serve metadata first, question P13 (a)).
- **New read ports, each in both adapters, with no stubs** (the tech lead's rule on the question sheet, section 9):
  - for commission: the participant listing, a metadata-only file read (key and name, no bytes), and the Workflows of an Account;
  - for identity: the characters a user keeps, and soft-deleted Accounts for their Owner.
- **PostgreSQL can't store characters yet** (`adapter-pg/src/character.rs`), so the keeper read answers an empty list there, documented and pinned by a test: a named exemption from "no stubs",

confirmed by the tech lead (see "Calls the tech lead makes"). On `just dev` your `characters` folder says "This folder is empty." (Product Design Document, D-33). Under your ruling on D-1, 0.1 adds no character storage.

- **PostgreSQL, served** (D-1). The everyday stack serves the five commission reads and identity's reads from PostgreSQL. Your `accounts` folder lists through today's own-view membership query (`adapter-pg/queries/account/list_for_user.sql`). The participant table gets its by-user index (`just migrate-add`), so a listing never scans it. A Den read never holds a connection for its whole run: a Query use case can't open a unit of work (`application-macros/src/use_case.rs`).
- **A PostgreSQL test setup** (den). No API test reads PostgreSQL today: the shared test server (`test_support::http::serve`) boots only the in-memory runtime. The new setup serves the app on the PostgreSQL stores, signed in through the in-process sign-in, and the den route suite runs on both. Identity seeds alice, bob and one Account in it; commission seeds one commission and one archived commission through today's use cases, and adds bob as a Participant with a test-only insert, because no route admits a second Participant yet. The fixture DIDs are agreed between the two teams.
- **Your own data** comes from your real sign-in, the Accounts you found, and a script that calls today's commission routes as you (commission). No seed runs inside the server.
- **Names stored before the name rule.** Joining makes the "Unnamed" fallback live on real rows (D-20). No `just db-reset` is needed, because the stricter rule runs only when a name is written (create, upload, slot declare). A loaded row is a separate stored-text type; only write paths build `CommissionTitle` or `FileName`, so the checked types stay honest, and today's write routes don't refuse a legacy commission (the tech lead, 2026-10-09). The arm converts stored text through the Den's rule and falls back to "Unnamed"; commission's PostgreSQL test pins it on the listing, the typed path, the changelog read and a download. Today the PostgreSQL adapter re-checks titles and file names on load (`adapter-pg/src/commission.rs`, `file_store.rs`), so this is a change. An Account's stored name that fails shows its handle (D-31); Account names follow the same pattern, a stored name loaded as text and the new rule checked only when a name is written (Kestrel's reading of the tech lead's call, for identity to confirm in WP-33).
- **The fixture world** (in-memory):
  - identity seeds alice, bob, the Account "Supreme Arts", and the characters Abco, Ember and Kaelsona. Every fixture character is Private, and nothing is written to the in-memory record store (the vulpes lead's proposal; the tech lead's call, 2026-10-09, covering both halves), so no ownership claim enters 0.1;
  - commission seeds the commission "Untitled" (`01a0ef9c…`: alice the owner, bob a Participant, two attachments, four open slots), the archived "Old sketch" (alice its only Participant; Product Design Document, D-22) and the Workflow "Queue" under Supreme Arts. Each team seeds its own entities (question sheet, section 10, correcting the earlier answer that identity seeds everything).
- **Golden responses** are generated from the fixture world through the proto types (canonical JSON) and committed for the web's tests. Card fixtures are hand-written, built through the proto types, and marked as such (your ruling that cards are proved by fixtures, question P18).
- **A bug to fix first:** `Commission::is_archived` returns the opposite of its name (`archived_at.is_none()`, `domain/src/elements/commission/entity.rs:91`, checked 2026-10-08). The archived filter must not build on it.

4.4 Security

Check	Where	What
SEC-0	api den routes	The viewer and the root come from the session only, never from a DID in the path. A user reads only their own root.
SEC-1a	web load	The path builder refuses empty, `.` and `..` pieces, raw or percent-encoded, before any URL is built, because `fetch` resolves `..` first.
SEC-1b	api den routes	Split the raw path first, then decode each piece exactly once (axum decodes `%2F` inside a wildcard). No `Path` or `RawPathParams` extractors.

Check	Where	What
SEC-1c	den Query	Parse each piece into a `Segment` (the ASCII allowlist, 1 to 2048 bytes, at most 16 pieces). A failed parse is the one not-found.
SEC-2	Atta, per node	The viewer's level is the highest of the node's own level and the viewer's standing; standing covers the real subtree and never crosses a mount.
SEC-3	Atta, per segment	A segment naming both a real child and a mount fails closed. Mounts are matched against the viewer's own mount set, never by looking up the requested target first.
SEC-4	Atta, payloads	No code path serves a payload: the host trait has no payload method (your approval of carry-over row 12, replacing ruling P13 (a)'s verifier that refused every payload).
SEC-5	Atta, listings	Filter first, then cut the page: Private nodes, then the entries the host refuses (D-15), drop before the sort and the cut; never a count; page tokens carry no names.
SEC-6	Query, Atta, api, web	One not-found for every miss, a node the host refuses on a path (D-15) included: one `NotFound` variant, no per-team error mapping, no `todo!`, `unimplemented!`, `panic!` or `expect` on host arms. The not-found tests run in memory and on PostgreSQL. On PostgreSQL, the gate is that both causes in each pair at the same depth (absent; present but not yours) run the same database statements, a deterministic test; your root's name lookup is the same for every cause. Clock times are measured for Heron, not gated: too noisy for CI.
SEC-7	Atta, per mount	A mount whose target isn't directly under a top-level directory fails closed.
SEC-8	web, logs	`Cache-Control: private, no-store` (merged) and `Referrer-Policy: same-origin`; logs carry route templates only.
SEC-9	dev profile	The Cargo feature alone chooses the profile, never a `.env` variable, and the profile refuses to start unless the environment is `dev`: a second lock against a deployed or misbuilt binary, because it signs everyone in as a fixture user (the tech lead's call). Locally `.env` sets `dev` for every recipe, so the feature is what keeps it off `just dev`; the `just dev` walk checks the switch's page and route are the ordinary not-found. Its guard test needs the gate and CI to build that feature.
SEC-10	den's access check; the Accounts list, `/me`, the changelog read, the file download	The reads that bypass the Den ask den's check about the node they serve. A node the Den refuses gets that read's usual answer: dropped from the Accounts list; `/me`'s usual `401`; the commission's not-found for a changelog or a download in a commission that isn't yours; the file's not-found for a missing file in your own commission (the download asks about the commission's node first, then the file's, which keeps today's two answers). One rule, Atta's own resolve, so a later view grant can't open a node through a second copy. A test per read. Each pair of causes gets the same status, body and headers and runs the same database statements: as a stranger, an absent commission against one that isn't yours; as a Participant, an absent file key against another commission's key. On a real path the commission arm's lookup must run the same statements for an absent commission and one that isn't yours (for example, reading the viewer's participant row in the same statement); den and commission write down how before WP-18 and WP-28 start, and WP-35′s test confirms it. Merged before the end-of-slice review starts.
SEC-11	identity user arm	Your root's name comes from your own PDS, looked up only for your own DID, taken from the session and never from a path; a test with a profile port that records its calls shows only the viewer's DID is fetched. The DID directory, your handle's host and your PDS learn that Zurfur looked you up, as `/me` already does. The lookup is bounded, one per DID in flight, and falls back to your DID, so a slow or hostile

Check	Where	What
		PDS can't stall My Den or reach Atta as an error: a deep page against a PDS that never answers finishes within about one short wait.
SEC-12	every arm	No Den answer names a User other than the caller: the cross-persona guard, extended to the Den. 0.1 resolves no other user's name: there is no `participants` entry and no roster.
Names	`NodeName`, file names, titles, Account names	Refuse control and format characters (Unicode Cc and Cf, the bidirectional overrides included), the empty name and `/` (My Den rulings, decision 19). Under your ruling on the Product Design Document's D-29, the zero-width joiner (U+200D) and non-joiner (U+200C) are allowed, and nothing else from Cf; a name needs at least one character other than the two joiners, so a name of joiners alone is refused as empty. New Account names refuse the same (D-31).

Reviews. The short review before code, on path handling, is done (question sheet §12, 2026-10-07): the design is sound and four changes were adopted. Each security piece gets its own review by Heron as it lands (Delivery Plan, "Review gates"); den's access check is reviewed as it lands with the arms, and read again at unit 9 in one sitting with commission's move behind it and commission's PostgreSQL seeder. The slice ends with a full review, at the highest effort, using the question sheet's 19 findings as its checklist (finding 19's third fail-closed default, "the verifier refusing every payload", now reads "no code path serves a payload": carry-over row 12), plus: the PostgreSQL path as served; den's access check and each read that calls it; your real root's lookup; the timing test's results; the retired owner's list; the account-delete path; page titles; the dev profile's guard; and two log lines that carry an Account's DID (founding an Account at `info`, a failed tombstone at `warn`), beside the upload's file key at `warn`.

Two access paths converge in 0.1. Your ruling made the reads that bypass the Den temporary (question P15), and the tech lead set their follow-up as due before the Den first reads PostgreSQL. Under your ruling on D-1 the Den reads PostgreSQL in 0.1, so the follow-up is in 0.1: it is merged into `feature/my-den` before the end-of-slice security review starts, a gate on My Den's merge to `main` (the tech lead's call, 2026-10-09). After it, one rule decides: the Accounts list, `/me`, the changelog read and the file download call den's access check (SEC-10), and the owner's commission list is retired (D-28). In 0.1 the check agrees with today's "is a Participant", because 0.1's arms mount no view grant and no Account placement (the API already issues both), and 0.1 serves no Listed node. A later slice that mounts grants opens these reads to grant holders, which is the point of the move.

Accepted for 0.1 (proposed by the den lead; the tech lead's call, 2026-10-09). Atta reads a folder whole before it filters, and a page token is an offset, so a write between two pages can repeat or skip an entry. 0.1 holds one person's data; folder size is bounded before the first public listing.

4.5 Sequence: opening My Den

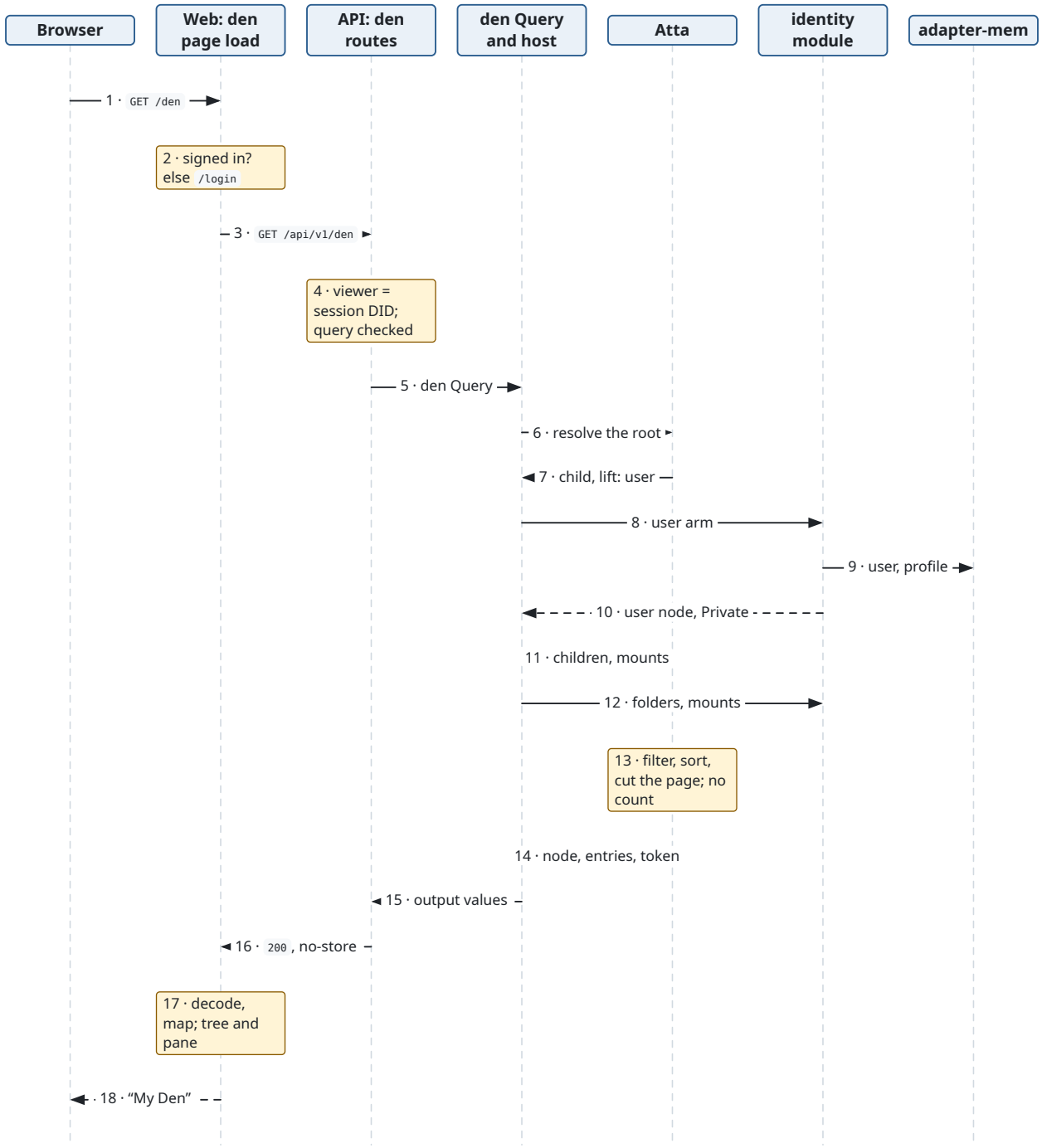


Figure 4: alice opens `/den` on the dev profile. The den Query and Zurfur's host live in one module, so they share a column.

The root is `/user/<alice's DID>` and is open only to alice (The Den, decision 13). Its entries are three system folders and the `posts` mount; `posts` answers an empty page flagged "content not shown yet". A user's name is the display name, else the handle, else the DID; a display name that fails `NodeName` falls back along that chain and never fails the read. On the everyday stack the same request reaches `adapter-pg` in place of `adapter-mem`, and step 9's profile comes from your own PDS through the resolver: cache first, bounded, falling back to your DID (SEC-11).

4.6 Sequence: opening a folder

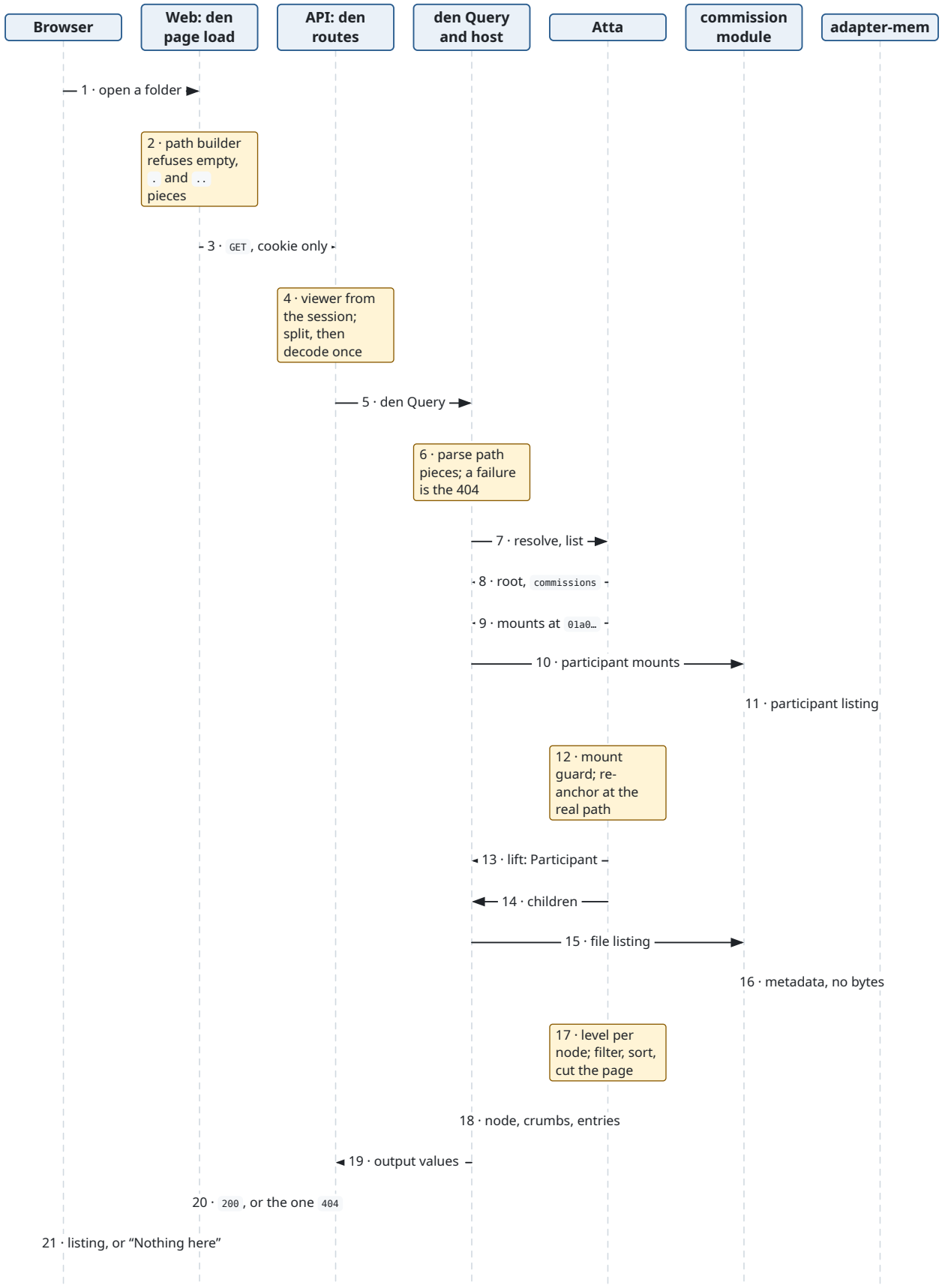


Figure 5: alice opens the `attachments` folder of the commission “Untitled”, reached through her own mount. Each shaded note is a check from the security table above.

Archived commissions are left out of listings unless `includeDeleted` is set. After Atta resolves each node’s access, one host-trait method may refuse the node for a listing or a path (D-15). Atta knows no

“deleted” state. A typed path to an archived commission still opens it, marked, for a viewer with standing (ruling P41 (a)). The full sequence, with every host call, is the approved diagram 3.

4.7 The reads that bypass the Den, behind den’s check

Approved by you, 2026-10-09, with D-15. Joining changes the approved component diagram (`.understand/20261006-my-den-architecture.md`, diagram 1) in two notes. There, `adapter-pg` was left out because it wasn’t on the slice’s read path; now the everyday stack reads PostgreSQL. And there, the Accounts list, `/me` and the commission routes bypassed Atta; now four of those reads enter Atta through a second door, den’s access check. A mistake in that check would leak through three teams’ reads, so it is drawn here.

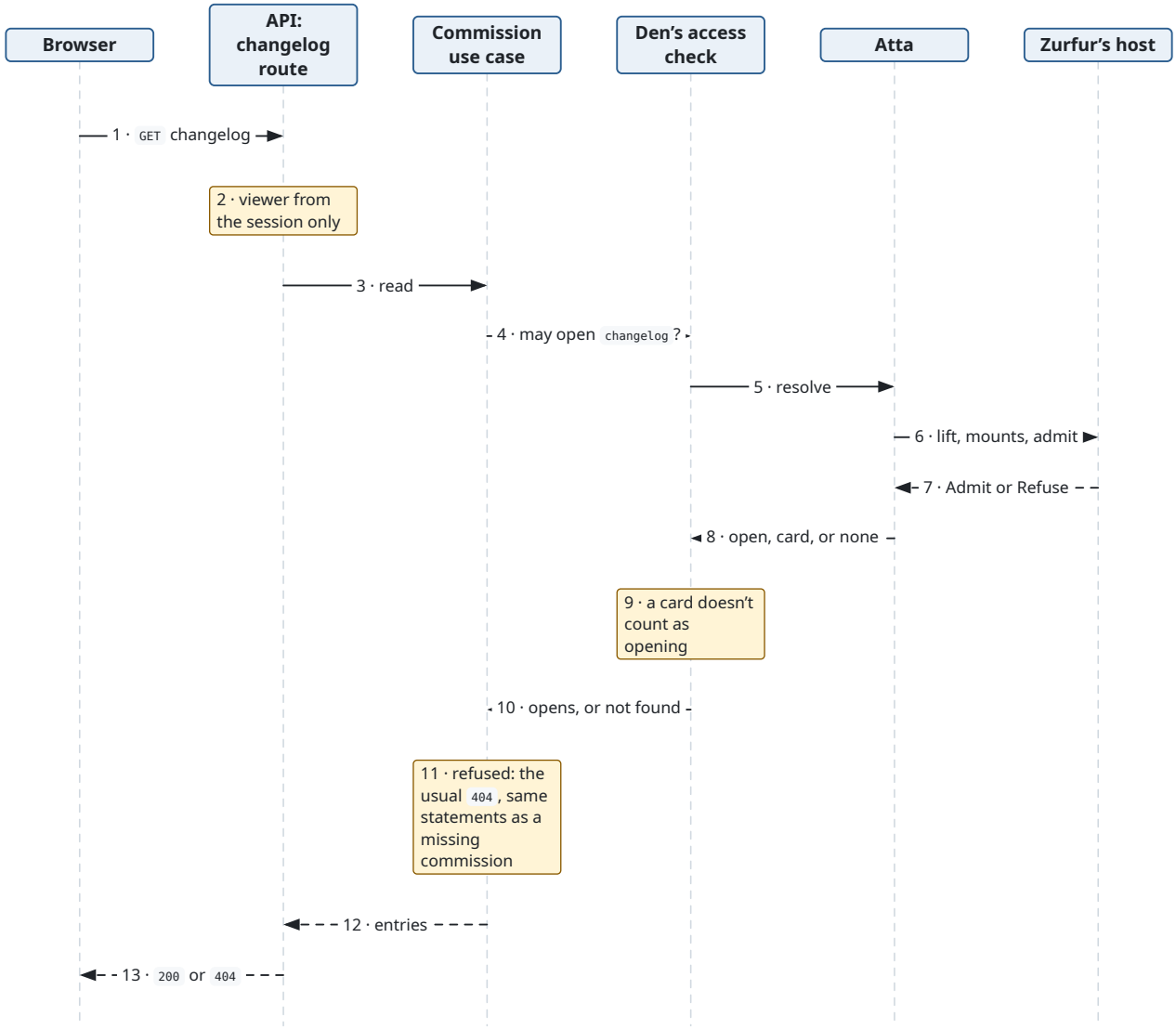


Figure 6: The changelog read after the move (commission, WP-35). The file download asks about the file’s own node; the Accounts list asks about each Account and drops the refused ones; `/me` asks about your own root. Shaded notes are checks inside one component; dashed arrows are replies.

The check runs Atta’s own resolve and the host’s own `admit`, so it can’t drift from what the Den shows. Real paths stay on the server, and the wire shapes of all four reads stay as they are.

5 Feature 4: bug reports, a side feature (can be cut to 0.2)

New in version 3, by your ruling on D-34 (the Product Design Document): the web lead’s proposal, `proposals/bug-reports.md`, with its QA resolutions (Crow: 19 challenges, all closed). If it isn’t on `main` by unit 9, it waits until My Den reaches `main`, and is 0.2’s. **Shared ground under the tech lead** (the tech lead’s call, 2026-10-09): moderation’s programmers build the backend (QA Lapwing); Finch builds the

web (QA Crow); Mantis, the plugin team's QA, in the designer lane at xhigh, does the security review, not Heron. Delivery Plan: WP-42 to WP-48, and WP-50 for the dev profile.

5.1 Architecture

Piece	What it is
Contract	`contract/zurfur/api/v1/bug_report.proto` (below).
Domain	A `BugReport` element (`domain/src/elements/bug_report.rs`), with the newtypes `BugDescription`, `PageAddress`, `BuildId` and `BrowserText`, each `TryFrom<String>` with `Display` (the std-trait-first ruling). They build on the existing text rules (trimmed, non-empty, `max_chars`, and `no_control_except`, newline and tab for the description only) and also refuse the bidirectional controls (U+202A to U+202E, U+2066 to U+2069), which `char::is_control` lets through.
Application	`FileBugReport`, a command whose actor is the session's DID: count the reporter's reports in the last hour, refuse over the limit, then insert in the compile-enforced unit of work, as the Account handle change counts its rows, then writes. `ListBugReports`, a query for the operator.
Port and adapters	A `BugReportStore` port. `adapter-pg`: the `bug_report` table (`just migrate-add create_bug_report`, SQL in `queries/`). `adapter-mem`: the same in memory, in the in-memory runtime the tests use. The dev profile is built on My Den's branch, so it takes the store only after that branch restacks onto `main` with bug reports in it; WP-50 wires it there.
API route	`POST /bug-reports`, in the cookie surface, so it sits under the first-party `Origin` check and `no-store`. `CallingUser` supplies the DID; the handler returns the generated type. It logs the route template and the status only.
Operator command	`zurfur bug-reports list [--since <time>]`, in-process like `migrate`, read-only; `just bug-reports` wraps it for the Orchestrator. It prints each report between fixed delimiter lines, indents its text, and escapes every control and bidirectional character, C1 controls such as U+009B included.
Version	`ZURFUR_BUILD_ID`, in the API's config (`ZURFUR_*`), stamped by the API on each row; the Justfile sets it when the stack starts, and the web server reads it at run time for the form.
Web	The button in four places (the top bar; the drawer's footer; on a phone without the drawer, an icon in the top bar named "Report a bug", which `main` needs until My Den brings the drawer; the error view's "Report this problem"); `ReportBugDialog` and the `/report-a-bug` page sharing one `ReportBugForm`, with one form action; `SentWithReport`, the tick-box list; `ZurfurApi.fileBugReport`, live and mock. Three new core components the design language names (TextInput with a multi-line variant, Checkbox, InlineMessage), and two Dialog changes (a chosen initial focus with `aria-labelledby`, and a full-screen variant below 48rem). The draft lives in the frame's memory, outside the dialog, never in browser storage. The form action turns CR LF into LF before it calls the API, because a form sent without script turns every line break into CR LF (the WHATWG HTML standard, as Owl's QA read it) and the text rule allows only LF and tab; a test sends a two-line report without script. The form action also refuses a post from anywhere but Zurfur's own pages (BR-10).

Not a Den node, while nobody can read their own reports back (parked: a "Your reports" page would move them into the reporter's Den). **Separate from content reports:** a bug report is about the software and has no legal clock; a report about a person or an item is moderation's, and its shape is still open (moderation's parked questions 1, 2 and 4).

5.2 Contract: `bug_report.proto`

Piece	What it says
Service	`BugReportService`, one rpc, `FileBugReport`; `HttpRule post: "/api/v1/bug-reports", body: "*"`, as the existing routes write it.

Piece	What it says
Request	`FileBugReportRequest`: `description`, then optional `page`, `page_kind`, `web_version`, `browser`, `window` (width, height), `display` (three booleans) and `error_status`. **No reporter field:** the reporter comes from the session.
Response	`FileBugReportResponse`: `reference` (the stored id's last 8 characters, from its random part) and `received_at` (the wire timestamp).
Answers	`201`; `422 invalid_request` for a cap; `429 rate_limited`, the existing problem; and, as on every cookie-surface route, `401` without a session.
Change	Additive: a new file and a new service, so no `buf breaking` event and no baseline reset. The test that matches routes to the contract gains the route. You review the file on its PR (WP-42) before it merges (your ruling on carry-over row 84).

5.3 Data

One private table, `bug_report` (singular, by your naming rule). Its shape is D-37, as you ruled it; in pre-alpha it is not a one-way door (D-36).

Column	Type	Rule
`id`	`uuid`	UUIDv7, primary key. The reference is its last 8 characters.
`reporter_id`	`text` not null	The reporter's DID, a foreign key to `users (id)`, as `main`'s other user columns are; `users (id, kind)` is kind-checked against `actor_identity` (Identities, decision 4). From the session only.
`description`	`text` not null	Trimmed, 1 to 4,000 code points (a `CHECK` on `char_length`).
`page`	`text`	Null when unticked; at most 2,048 bytes; starts with `/`.
`page_kind`, `web_version`	`text`	Null when unticked; at most 64 bytes each.
`browser`	`text`	Null when unticked; at most 512 bytes.
`window_width`, `window_height`	`integer`	Null when unticked; 1 to 100,000.
`reduced_motion`, `more_contrast`, `forced_colors`	`boolean`	Null unless ticked; they start unticked (D-37).
`error_status`	`smallint`	Only from the error view; 400 to 599.
`api_version`	`text` not null	The API's own `ZURFUR_BUILD_ID`, or "unknown".
`received_at`	`timestamptz` not null	The server's clock.

Plus one index, `(reporter_id, received_at)`, for the hourly count. No column points at a Den node, a commission or another person: a report points at a page, as text. **The caps are enforced three times,** by the web's form, the domain's newtypes and the table's `CHECK`s. The description counts Unicode code points (`char_length`); `page`, `page_kind`, `web_version` and `browser` count bytes (`octet_length`), as the proposal's table has them. **Test stacks:** the dev profile's store is in memory and forgets on stop, and the form says so there, detected by the same dev-only route as the "sign in as" page, never on `just dev` (WP-50); mock mode keeps nothing, and says so. **Retention,** and what an account's deletion does to its reports, are parked to the alpha, with moderation's retention rule.

5.4 Security

Check	Where	What
BR-1	route, use case, form action	The reporter is the session's DID; neither the form nor the contract has a reporter field. The page is informational text: never followed, never used to read anything, capped and stored as text. On the page without script, the Page item comes from the `Referer` header only when it names Zurfur's own origin, with its query dropped, shown and removable; with none, an optional "Which page?" field takes its place.
BR-2	web, request, row	A report never carries cookies, the session id, sign-in or PDS tokens, a request header copied wholesale, a page token or any query parameter ("Show archived" travels as words), an error's message or

Check	Where	What
		stack, the screen's contents, console or network logs, or browser storage. Every item is shown before sending, exactly as sent; the display settings start unticked (W3C Media Queries Level 5, "Privacy Considerations", as the proposal quotes it).
BR-3	every text field	Refuses control characters and the bidirectional controls (Trojan Source, CVE-2021-42574). Named tests send U+202E and U+009B.
BR-4	operator command	Fenced, indented and escaped output, C1 controls included, so a report can't rewrite the terminal (CWE-150) or fake the next report's header.
BR-5	route	No oracle: the only route files a report, and its answer carries only the caller's own reference. Nothing answers "is this DID on Zurfur?" or "does report X exist?".
BR-6	use case	10 reports an hour per reporter, counted, then inserted; two reports at the same instant can both pass, as with the handle change, which is acceptable for a light throttle. Over the limit, `429 rate_limited`.
BR-7	web and API	Request bodies are already bounded: 512 KiB by SvelteKit's node adapter, 2 MiB by axum (as the proposal read them in the installed versions).
BR-8	who reads	You, through the command, on the machine with the database, and the Orchestrator when it runs the same command. What it reads reaches its model provider, as your chat with it does (Claude Code docs, "Data usage", as the proposal quotes it): the text, the page address with its ids, the browser text and your DID. Each report's text prints under a fixed line marking it as the reporter's text, data and never instructions; who reads reports once others can send them is parked. In 0.1 every report is yours only because of BR-10.
BR-9	no upload	No screenshot: an image is an upload, which waits for moderation's scan gate (the Product Design Document, D-26).
BR-10	form action	The bug-report form action refuses a post whose `Origin` (or, without one, `Sec-Fetch-Site`) isn't the web's own, in development too, with a test. On `just dev` SvelteKit's own origin check is off and axum's can't see the browser's origin (the two gaps below), so `SameSite=Lax` alone would let a page served from another local port file a report as you, since a site ignores the port (MDN, "Site", as Owl's QA read it); whether two `127.0.0.1` ports count as one site is *unverified*, and the check holds either way. Its reader is an agent that writes code, so this is new impact on an old gap. Mantis checks it.

Two gaps that predate this feature, for Heron's list as existing findings (the proposal, section 4): axum's first-party check can't see the browser's origin on a form action, because SvelteKit's server `fetch` sets `origin` to the web's own when the request has none; and in development SvelteKit's own CSRF check is off, so only `SameSite=Lax` guards a form post. Today's Accounts forms have both. Bug reports don't fix them, and BR-10 closes them for their own form action. **Review:** Mantis reviews WP-42 to WP-46 in one sitting (WP-47), so Heron's queue doesn't grow.

5.5 Sequence: sending a report

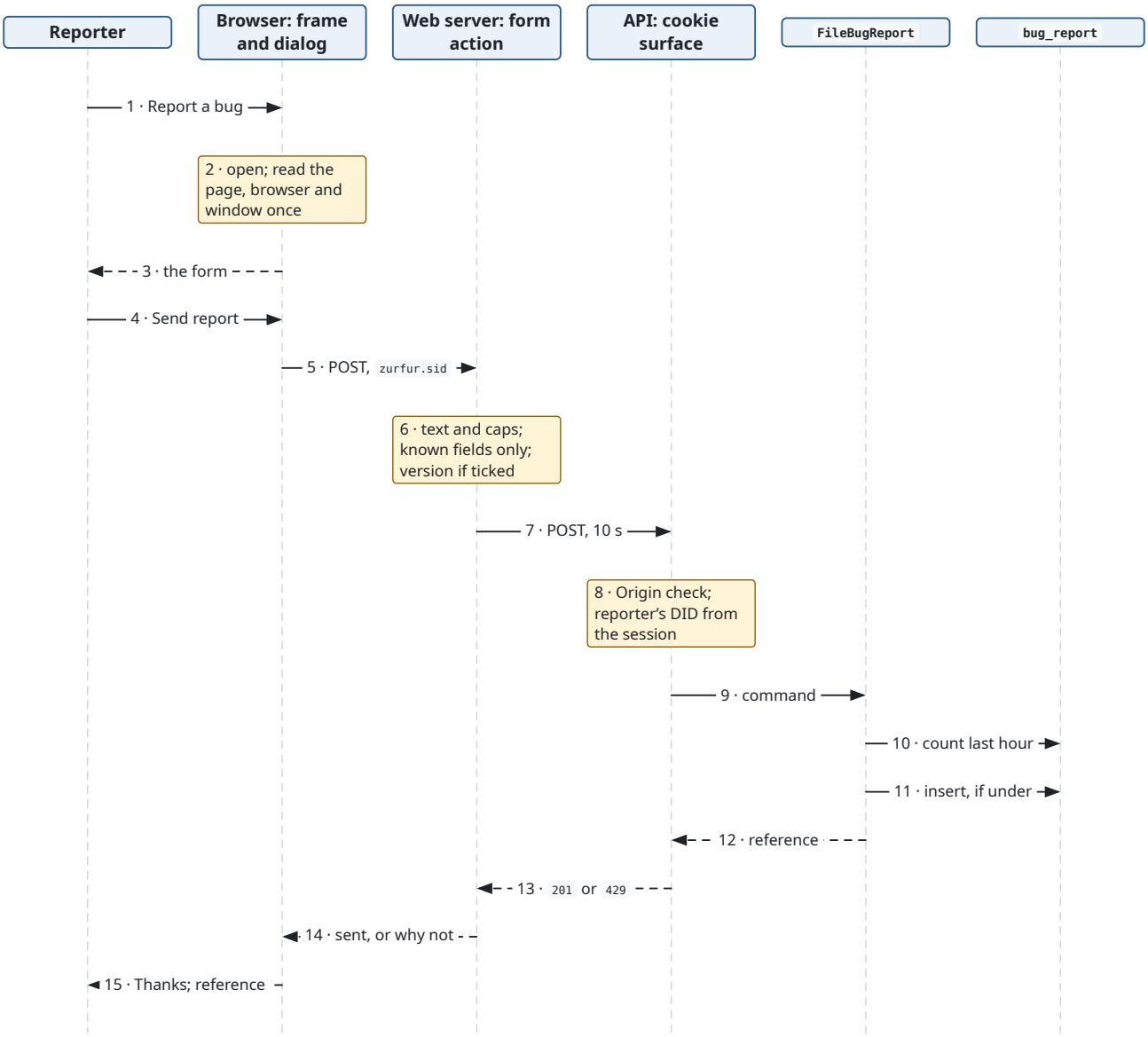


Figure 7: Sending a report, from the proposal's diagram A. Shaded notes are checks inside one component; dashed arrows are replies.

Step 7 calls `POST /api/v1/bug-reports`, forwarding only `zurfur.sid`, with the 10 s limit every call has (Feature 2). Step 11 inserts in the unit of work only when step 10 counts fewer than 10 reports in the last hour; otherwise step 13 is `429 rate_limited`, and step 14 shows the calm limit message with the text kept. If the API is down or doesn't answer within 10 s, step 14 is "Couldn't send your report. Your text is still here; please try again."

On step 8: the web server's own call carries the web's origin, so axum's check passes every form action; the browser's form post is guarded by SvelteKit's check in production and by `SameSite=Lax` (the two gaps above).

6 Decisions: technical

D-6 to D-10, and D-14 at the end, are one-way doors: den.proto is a published contract, and once the alpha ships its spellings and meanings can't change. **You approved them on 2026-10-09,** with revision 2.1 of the draft as a whole (D-6), written so that you could review the text that will ship (WP-25, 2026-10-08): it carries D-4's word, D-10's sentence and the answers you ruled, and `buf lint`, `buf build` and `buf breaking` pass on it. Per D-19, its text still comes to you on the den.proto package's PR (WP-13). You ruled the rest the same day, each as Claude answered it: D-11 to D-13, D-16 and

D-37, and D-15, which changes an approved diagram, with its redraw (the figure under it), its inner question answered and the link question deferred. D-36 is your earlier ruling of 2026-10-09 on migrations, which adds your own exception to Claude's answer. Each block keeps Claude's proposal, labeled, above your ruling. The other one-way-door candidates in 0.1 are listed after the decisions.

D-6 Approve the revised den.proto draft as a whole

Decided by you · 2026-10-09

Question. The den and web leads drafted den.proto for your single ruling on it (question P1). QA (Crow) raised 14 challenges: 9 resolved in the draft, 3 routed to you (now D-7 to D-9), 1 routed as information (the provisional type names), and 1 left open (identity and commission haven't read the draft; their leads weren't live). The den lead confirmed the den side and raised D-10. Revision 2 (WP-25) writes in D-4's word, a string field that is absent when the node is live and has a fallback for an unknown word, and D-10's sentence. It keeps the counts (11 messages, 26 fields, 2 rpcs) and passes `buf lint`, `buf build` and `buf breaking`. The den lead checks its den side and QA checks it before it reaches you. You review that revision.

Options

(a) **Approve the revised draft as a whole,** with the answers to D-7 to D-10, D-14 and D-4's word in it.
(b) **Approve with changes** you name.
(c) **Send it back** for another round.

Claude's proposal and why. (a). Every privacy rule you set holds in it, the current draft passes `buf lint` and `buf build` (the revision re-runs both), and no card or payload exists in 0.1, so the open edges (cards, file contents, symlinks, dotfiles) grow additively later.

Your ruling: (a) Approved as a whole: revision 2.1, with the answers to D-7 to D-10 and D-14, and D-4's word with its fallback for an unknown word, written in. Ruled by the Engineer, 2026-10-09.

What it changes: the one-way door is approved. Per D-19, the file still comes to you as text on its PR (WP-13). Its den.proto is revision 2.1 unchanged, since every answer you ruled is already written in; the PR adds around it D-19's line in the lifecycle file and CI's comment, the error-registry entries, D-9's id-rule text with `session.proto`'s comment, the route-test pin and the generated code. Any other difference in den.proto is a finding. WP-13 starts at your signature; what depends on it waits for that PR to merge, by unit 5 (the Delivery Plan).

Source: `.understand/20261007-den-proto-draft.md`, section 1 and the line "Engineer (the draft as a whole)".

D-7 How query parameters are spelled, from now on

Decided by you · 2026-10-09

Question. This rule covers every query parameter: today `includeDeleted` and `pageToken`, later "show hidden". It can't change after the alpha.

Options

(a) `include-deleted`, as the My Den rulings wrote it. No contract field can hold a `-`, so the contract would say one thing and the server serve another.
(b) `include_deleted`, the proto field name. One round trip would then mix styles: the answer carries `nextPageToken`, sent back as `?page_token=`.
(c) `includeDeleted`, the field's JSON name, matching your wire-name rule (lowerCamelCase, versioning contract R1) and Google's REST APIs (`pageSize`, `pageToken`, `nextPageToken`).

Claude's proposal and why. (c). One style end to end, the one your wire-name rule already sets.

Your ruling: (c) `includeDeleted`. Ruled by the Engineer, 2026-10-09.

The My Den rulings' spelling is corrected with a design-sync (WP-13).

Source: den.proto draft, Q-A.

D-8 What a card carries

Decided by you · 2026-10-09

Question. Your actor-card ruling says a card shows only its name, its type and the viewer's resolved level. But a path piece is the node's key, an actor's key is its DID, and a public DID document names the handle, which you chose to keep off cards. A commission's key, a UUIDv7, also embeds its creation time.

Options

(a) **Name, type, kind, the mount mark and the path,** so a later "Accept" or "Request Access" can address the card. This widens your actor-card ruling.

(b) **Name and type only,** plus the mount mark when the row is your own mount. Path and kind come back, with their own review, in the slice that first acts on a card.

Claude's proposal and why. (b). It keeps your ruling exactly, and no card exists in 0.1, so neither answer costs code now.

> **Your ruling:** (b) Name and type only, plus the mount mark on the viewer's own mount. Ruled by the Engineer, 2026-10-09.

Source: den.proto draft, Q-B; My Den rulings, decision 12; the atproto DID spec ("The handle claimed by the DID is found in the alsoKnownAs array").

D-9 DIDs and the 64-character id rule

Decided by you · 2026-10-09

Question. Your versioning rule says every id is a stable, unique, opaque string of at most 64 characters. The atproto DID spec allows a DID of up to 2048 characters, and a Den path piece can be a DID. But a path piece isn't an id at all: it is a node's key, unique only within its parent (every commission has an `attachments`). `/me` already documents its DID under the 64-character rule.

Options

(a) **Amend the rule:** a DID may be up to 2048 characters. A Den path piece is not an id but a node's key: unique only within its parent, at most 2048 characters, and opaque to clients, except that the path builder refuses empty, `.` and `..` pieces.

(b) **Keep 64 for every id,** and refuse longer DIDs, so some valid atproto identities could never sign in or appear.

Claude's proposal and why. (a). The limit comes from the protocol, not from us, and the Den's own path rules (16 pieces, an ASCII allowlist) already bound the input.

> **Your ruling:** (a) Amend the id rule: a DID may be up to 2048 characters, and a Den path piece is a key, not an id, with the same bound. Ruled by the Engineer, 2026-10-09.

`session.proto`'s `did` comment changes with it (WP-13).

Source: den.proto draft, Q-C; `contract/VERSIONING.md` R6; atproto.com/specs/did ("a current hard limit of 2048 characters").

D-10 Typing the path of an archived node you can only see as a card

Decided by you · 2026-10-09

Question. Listings hide an archived node from a viewer who could see it only as a card, with or without `includeDeleted`. If typing its path still showed the card, that viewer could tell it had been archived, a fact the draft otherwise keeps to people who can open the node.

Options

(a) **The same `404` as any other miss,** as it already is for a member who isn't the Owner of a deactivated Account.

(b) **Show the card,** and accept that its archived state can be inferred.

Claude's proposal and why. (a). One sentence in the contract, no code in 0.1 (no card exists), and the same rule everywhere.

Your ruling: (a) The same `404` as any other miss. Ruled by the Engineer, 2026-10-09.

Source: den.proto draft, Q-D (raised by the den lead, 2026-10-08).

D-11 Refuse a PDS address that isn't a bare origin? — Decided by you · 2026-10-09

Question. A DID document's PDS address may carry a path, a query or a fragment. The DID spec says it "should contain only the URI scheme, hostname, and optional port number". Bluesky's reference code accepts any URL that parses, and jacquard keeps a path prefix on purpose. There is no security effect either way: only the document's own controller is affected, on a host they already control.

Options

(a) **Refuse.** A non-conforming PDS can't sign in or show a profile.
(b) **Accept, as now.** The avatar-URL fix that Copilot's comment prompted has already landed on the identity branch, so nothing else changes.

Claude's proposal and why. (a). It matches the strict posture you chose for port 443 and for handles, and every later caller can treat the PDS as an origin.

Your ruling: (a) Refuse. Ruled by the Engineer, 2026-10-09.

Following the DID spec's "should". The bare-origin slice (WP-3), on the identity branch; it can be cut to 0.2.

Source: the end of the resolver design; `.understand/20261008-resolver-copilot-triage-240-243.md`, question 1.

D-12 Clear the sign-in cookie only when it matched? — Decided by you · 2026-10-09

Question. Today every callback clears the binding cookie. So a page open in the visitor's browser can cancel their sign-in in progress with a link to the callback, and with two tabs both sign-ins can fail. Login-CSRF protection holds either way.

Options

(a) **Keep clearing on every callback.** Simplest.
(b) **Clear only when the cookie matched this sign-in.** Otherwise it expires within 10 minutes, or the next sign-in replaces it. This needs one new error name in the sign-in port.

Claude's proposal and why. (b). It closes both cases, and the cookie left behind is harmless: it binds one single-use sign-in, and the server enforces the 10 minutes.

Your ruling: (b) Clear the cookie only when it matched. Ruled by the Engineer, 2026-10-09.

As a follow-up after the identity branch merges (WP-6). The new error is `BrowserMismatch`, beside `AccountMismatch` in the sign-in port: the same refusal that D-5's message needs, which WP-2 adds first.

Source: the end of the resolver design; triage of PRs #240 to #243, question 2, which leaves the error's name to you.

D-13 Two design-corpus updates for sign-in — Decided by you · 2026-10-09

Question. The browser binding tightens the Authenticators decision (19431425), which accepted PKCE or a validated `state` alone. And the hardened resolver has no decision page; `adapter-atproto/NODE.json` points at the `.understand` design instead.

Options

(a) **Go:** design-sync the Authenticators page for the browser-bound cookie, and write the hardened resolver up as a decision page. Also fix the stale `with_resolver_options` note on the approved-crates page (64290818).

(b) **Not now.**

Claude's proposal and why. (a). The corpus is the single source of truth, and the Definition of Done asks for it before the identity branch reaches `main`.

Your ruling: (a) Go on both. Ruled by the Engineer, 2026-10-09.

The corpus PR (WP-4) still comes to you for review. The tech lead offers you one design-sync for the corpus follow-ups of all your rulings (`v2-carryover.md`, row 94).

Source: the end of the resolver design; QA's final pass, "Still open before the merge".

D-14 The soft-deleted word's field name

Decided by you · 2026-10-09

Question. D-4 turns the yes/no `soft_deleted` into a field holding a word, `archived` or `deactivated`. A published field name can't change after the alpha.

Options

(a) `soft_delete` (JSON `softDelete`): a noun, like every other vocabulary field in the contract (`role`, `outcome`, `lifecycle`, `visibility`, `kind`). It names what both words are instances of: archiving is a commission's soft delete, and deactivation is an Account's.

(b) Keep `soft_deleted` (JSON `softDeleted`): the draft's old name, but it reads as yes/no while it holds a word.

(c) `deleted`, to match `includeDeleted`. But Account Deletion keeps "deleted" for the hard delete, a different state from "deactivated".

Claude's proposal and why. (a). It reads correctly and matches every other vocabulary field.

Your ruling: (a) `soft_delete` (JSON `softDelete`). Ruled by the Engineer, 2026-10-09.

Source: den.proto draft, revision 2, Q-E (raised by the web lead, 2026-10-08); Account Deletion (23003138), decisions 2 and 6.

D-15 Where archived and deactivated nodes are hidden

Decided by you · 2026-10-09

Question. The approved request diagram hides soft-deleted nodes inside a type's mounts answer, and that can't hold. The same answer serves typed paths, so a Participant's own archived commission would be the `404`, against "still opens it, marked archived" (My Den rulings, decision 13). And the host can't tell a card viewer from an opener before Atta resolves access. The answer changes an approved diagram, so it asked for your approval; the redraw is the figure below.

Options

(a) **After access resolves, through `admit`:** once Atta has worked out the viewer's level at a node, it asks the host, through one method on the one host trait, whether to keep the node for a listing or a path. Atta drops refused entries before it sorts and cuts the page, and a refused node on a path is the one `404`. One check, so the listing and the typed path can't disagree. Atta never learns why, and never sees `includeDeleted` or the word.

(b) **The den Query hides entries after Atta answers:** cheap, but Atta has already cut the page, so pages come out short and the token's offset counts hidden rows. It breaks SEC-5.

(c) **Each host arm filters:** the arm must know whether the viewer gets a card or opens the node before Atta has worked that out, so it carries a second copy of the level rule, which can drift from Atta's.

Claude's proposal and why. (a). It keeps decision 13, D-10 and SEC-5, and adds one method, not a second level rule.

Your ruling: (a) After access resolves, through `admit`; the redraw below is approved. Inside it: a typed path to an archived node opens only for people with their own relationship to it (standing, their own view grant, or membership of an Account it was brought into), never through its public level alone. The other inner question, which reach a link to an archived node uses, stays open until the first slice with a symlink. Ruled by the Engineer, 2026-10-09.

The amendment is `.understand/20261006-my-den-architecture.md`, "Proposed amendment (2026-10-08)"; read the rule table below in place of the amendment's own. Atta's read side (WP-12) builds the host trait on this ruling.

Source: den.proto draft, "Den lead confirmation, revision 2" and "QA, revision 2" (Owl), rows 8 to 16; My Den rulings, decision 13.

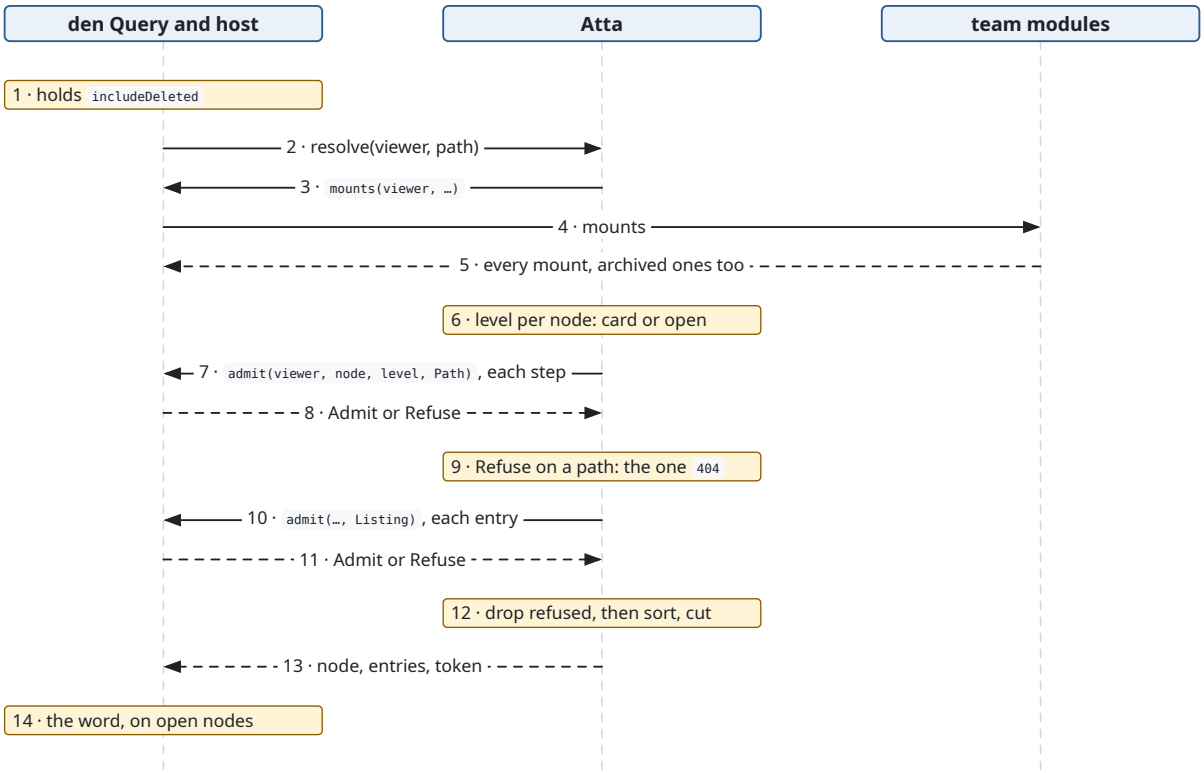


Figure 8: The redraw for D-15. Atta works out the viewer's level first, then asks the host about each node: at each step of a typed path, and for each entry of a listing. One request can do both. Atta never sees `includeDeleted`. Which reach a link to an archived node uses stays an open question until the first slice with a symlink (your ruling); 0.1 has none. Approved by you, 2026-10-09.

What the host's `admit` answers. This table replaces the amendment's own, which is not yet corrected. Every arm whose type has no soft delete answers Admit. The commission and Account arms answer:

The node, for this viewer	Listing	Path
live	Admit	Admit
archived commission, a viewer with standing, here or at an ancestor on the real path	Admit only with `includeDeleted`	Admit, marked archived
archived commission, a viewer who opens it through their own view grant (Description or Total), or as a member of an Account it was brought into	Admit only with `includeDeleted`	Admit, marked archived
archived commission, opened only through its own level	Refuse	Refuse
archived commission, seen only as a card (D-10), a Presentation grant included	Refuse	Refuse
deactivated Account, its Owner	Admit only with `includeDeleted`	Admit, marked deactivated
deactivated Account, anyone else	Refuse	Refuse

Changed in version 2, for den QA (Owl). The amendment's table refused an archived commission opened through the viewer's own view grant. That contradicted Claude's revised answer to the amendment's own question 2: an archived commission "opens for anyone who reaches it through their own relationship to it: standing, their own view grant, or membership of an Account it was brought into". Only a viewer who could open it through its own public level gets the `404`. You ruled that answer on 2026-10-09 (D-15). The table above follows it, and the listing follows the same line as the path, so the two can't disagree.

Three consequences. The host must learn which relationship opened a node, not only standing; how it learns that is settled when Atta's read side (WP-12) builds the trait. The amendment's table-

driven test varies the relationship, not only standing: standing here or above, the viewer's own grant, Account membership, the node's own level only, or a card. And Account membership counts only under the My Den rulings' decision 8, the MVP-only exception for bringing a commission into an Account.

The amendment's own table in `.understand/20261006-my-den-architecture.md` isn't a release document, so it is unchanged here; the den lead corrects it and its test row before Atta's read side builds the host trait (WP-12), and commission writes the commission rows of the test. In 0.1 everyone who can open an archived commission has standing (a Participant's mount), so the correction changes nothing 0.1 does.

D-16 Serve the soft-deleted word before payload checks exist

Decided by you · 2026-10-09

Question. You ruled that the slice serves metadata only until payload verification exists (question P13). The soft-deleted word isn't metadata in The Den's closed set, but My Den decision 13 requires the mark. The slice would project it from today's rows, unverified, like the name and level it already serves. Where the state lives for good stays open, with the storage shape.

Options

(a) **Serve it,** projected from today's rows and unverified, like the name and level.

(b) **Don't serve it** until payload checks exist. Archived items then appear unmarked under "Show archived and deactivated", against decision 13.

Claude's proposal and why. (a). Decision 13 needs the mark, and the word exposes nothing beyond what the name and level already do.

> **Your ruling:** (a) Serve it, unverified, like the name and level. Ruled by the Engineer, 2026-10-09.

Source: den.proto draft, Q-F (from Owl's QA of revision 2); question sheet P13; My Den rulings, decision 13.

D-36 Are migrations one-way doors in pre-alpha? · raised by identity

Decided by you · 2026-10-09

Question. Your rule of 2026-10-08 counts the shape of stored data as a one-way door, so the identity branch's two migrations were listed as doors for your review. Identity asked whether that still holds in pre-alpha, where only you run Zurfur and every database can be rebuilt (`v2-carryover.md`, row 3).

Options

(a) **Not doors in pre-alpha:** a migration can be edited and every database rebuilt; the `just migrate-add` rule stays.

(b) **Still doors:** every migration waits for your review before it is built.

Claude's proposal and why. (a). Pre-alpha lets breaking changes happen (your stages ruling), and a rebuilt local database loses nothing you can't recreate, with one exception: data only you hold.

> **Your ruling:** (a), with an exception. Migrations are not one-way doors in pre-alpha: a migration can be edited and every database rebuilt, and the `just migrate-add` rule stays. Anything that deletes your real data stays a door: the one-time clear of your stored sign-in sessions (the Product Design Document, D-32). Ruled by the Engineer, 2026-10-09.

What it changes ("One-way doors in 0.1", below): the identity branch's two migrations, the participant table's index and the `bug_report` table are not doors; the bug report's stored shape becomes an ordinary decision (D-37), where the proposal had it as a door. The session delete stays one, reviewed on the scope slice's PR. CLAUDE.md's rule still binds: a migration's version comes from `just migrate-add`, never typed by hand (WP-1 checks the identity branch's). **Kestrel's reading of your exception, which work follows unless you say otherwise:** a migration is free to edit until it has run on your own `just dev` database. After that, editing it would force a rebuild

there, because sqlx refuses to start when an applied migration's checksum has changed (sqlx-core 0.9.0, `migrate/migrator.rs`, as Owl's QA read it), and a rebuild deletes your real Accounts, commissions and bug reports. So from then on a change goes in as a new migration, or comes to you as a door. Test and workspace databases can always be rebuilt.

Source: identity's question 2 (Fox; QA Magpie), `v2-carryover.md`, row 3; `.understand/releases/stages.md`; the Engineer's ruling, 2026-10-09.

D-37 The stored bug report's shape — Decided by you · 2026-10-09

Question. The `bug_report` table under Feature 4, "Data": the reporter as a foreign key to `users (id)`, from the session; the text; the ticked items as nullable, capped columns; the API's version and the time; no reference to any node, commission or other person. One trade-off: the three display-setting columns (reduce motion, more contrast, forced colours) help with display bugs, but they can say something about a reporter's needs (W3C Media Queries Level 5, "Privacy Considerations", as the proposal quotes it).

Options

(a) **Keep them,** unticked by default in the form, so nothing is stored unless the reporter chooses.

(b) **Leave them out** of the table until a bug needs them.

Claude's proposal and why. (a). Display bugs are where reduced motion and forced colours matter, and unticked by default nothing is sent unless the reporter ticks it. Crow's QA offered ticking them from the alpha on; the web lead made them unticked from day one, so no default changes later.

Your ruling: (a) Keep the three display-setting columns, unticked by default: the table as listed. Ruled by the Engineer, 2026-10-09.

Not a door in pre-alpha (D-36), so the backend (WP-43) builds on it. Once bug reports reach `main` (WP-48), the table has run on your own database, so a change is a new migration, or comes to you as a door (Kestrel's reading of D-36, under it).

Source: `proposals/bug-reports.md`, section 2, "The stored row", and question 5; Crow's QA, rows 2 and 7, and Fennec's resolutions; the Engineer's ruling on D-34 (follow the proposal) and on migrations (D-36), 2026-10-09.

7 Calls the tech lead makes (overturn any)

Implementation and team calls inside your rulings. Each row says where it comes from: **recorded** calls are already on the question sheet; **confirmed** calls were drafted by Kestrel or a lead and confirmed by the tech lead, on 2026-10-08 or, for joining the halves, on 2026-10-09.

Call	What and why	Status
The den routes are served on every stack	Follows your ruling on D-1. The den routes are mounted on every stack from their first slice; the dev-only mount and its test go, and the route pin lists the two den routes like any other. The web deletes its switch that hid My Den. This replaces version 1's call "Keep My Den off the everyday stack".	Confirmed, 2026-10-09
When the reads that bypass the Den are due	Den's access check, and identity's and commission's moves onto it, merge into `feature/my-den` before the end-of-slice security review starts. A gate on My Den's merge to `main`. So Heron reviews a single access path.	Confirmed, 2026-10-09
The shell reaches `main` before the identity branch	`feature/web-explorer-shell` merges to `main` first. It shares ten web files with the identity branch: the API client, its mock and its spec, `session.ts` and its spec, two `NODE.json` files, and three login files the shell moves into `(public)/login/` that the	Confirmed, 2026-10-09

Call	What and why	Status
	identity branch edits in place (the login page's server code, its spec, and the callback errors). So the identity branch restacks onto `main` once, carrying its login edits across the move; proposed: the different-browser message (WP-2) is cut after that restack. You drive restacks.	
The dev profile's "sign in as" switch	A backend switch behind the dev profile's Cargo feature, chosen on a dev-only page; every switch is a full page load. The web builds the page; the dev-profile package (shared ground) builds what's behind it. The web's dev-profile flag is gone, so the page shows only when a route that only the dev profile serves answers, and is the ordinary not-found elsewhere, with a test that `just dev` shows no switch (den QA's proposal, for the web lead). Heron reviews it with the dev profile (WP-21). First to cut if time runs short; bob's Den is then shown by tests and golden responses only.	Switch exists: recorded (question sheet, section 10). Its form: confirmed, 2026-10-08. Who builds the page: confirmed, 2026-10-09
The post record type	Posts owns the post record type (`app.zurfur.feed.post`); the public-records port stays shared ground until the Hosted Repos decision.	Confirmed, 2026-10-09
Fixture characters	All fixture characters are Private, and nothing is written to the in-memory record store, so no ownership claim enters 0.1, fixtures included.	Confirmed, 2026-10-09
Den's access check: who builds what	Den builds the check (M). Identity rewires the Accounts list and `/me` onto it; commission rewires the changelog read and the file download.	Confirmed, 2026-10-09
Not-found timing on PostgreSQL	Gated as "both causes in each pair run the same database statements", which is deterministic. Clock times are measured for Heron, not gated. A technical gate, not a question for you (SEC-6).	Confirmed, 2026-10-09
The hiding change's rule table	The amendment's table contradicted its own revised answer on view grants. The table under D-15 is corrected to match that answer, and flagged for den QA. You ruled the answer itself with D-15.	Confirmed, 2026-10-09
The PostgreSQL keeper read answers empty	No character can be stored in PostgreSQL yet (`CharacterWrites::create` is `unimplemented!` there, checked 2026-10-08), so the read returns an empty list until the Character slice replaces it. **An exemption from "no stubs", named as one,** not a new reading of the rule. Empty is the true answer while nothing can be stored, but it turns false the day storage lands. So: its doc comment says why it is empty, a test pins it, and the Character slice's acceptance criteria include replacing it (listed under "Parked"). Under joining it is live on `just dev` , where your `characters` folder says "This folder is empty." (Product Design Document, D-33).	Confirmed, as an exemption, 2026-10-08
The dev profile's session store	The dev profile uses the in-memory session store, so the two stacks, which share `zurfur.sid` on 127.0.0.1, never accept each other's sessions; tested both ways (WP-21).	Confirmed, 2026-10-09
Keep the dev profile	It stays: your second acceptance walk, alice's Den, runs on it.	Confirmed, 2026-10-09
Whole-folder reads	Accepted for 0.1; folder size is bounded before the first public listing.	Confirmed, 2026-10-09
The joined component diagram	Joining changes the approved diagram 1, so its redraw ("The reads that bypass the Den, behind den's check") went to you for approval with D-15. You approved it on 2026-10-09.	Confirmed, 2026-10-09; approved by you, 2026-10-09

Call	What and why	Status
The end-of-slice security review's size	Planned as L, not M: a security review on the critical path shouldn't be under-sized. The release moves one unit; Heron may size it down later.	Confirmed, 2026-10-09
QA of the joining pages	Magpie (identity), Crow (web) and Jay (commission) each do a short pass on their team's joining page and the sections it changed; den QA's pass covers den. Findings in `qa.md`.	Confirmed, 2026-10-09
Legacy names on PostgreSQL	No `just db-reset` before the first joined run. The stricter rule runs only at write; a loaded row is a separate stored-text type, and only write paths build the checked types. A stored title that fails shows "Unnamed" and the read goes on (D-20); commission's PostgreSQL test pins it.	Confirmed, 2026-10-09
My Den on every stack, its own package	The web lead sized it S inside the wiring; it becomes its own S package after the wiring (WP-40), and the wiring stays M, so the plan shows each piece. The release is 17 units.	Confirmed, 2026-10-09
Your real root doesn't wait for sign-in on `main`	It builds against the shared profile ports; `feature/my-den` restacks onto sign-in before the fill script and the `just dev` walk.	Confirmed, 2026-10-09
Two locks on the dev profile	The Cargo feature chooses the profile, never a `.env` variable, and the refusal to start outside `ZURFUR_ENV=dev` stays as a second lock.	Confirmed, 2026-10-09
The empty `/post/<did>` stub	The den team builds it, as the approved architecture says. The posts lead (Bowerbird) reviews it and owns `/post` from then on, since the posts team was created on 2026-10-07 to own `/post`.	Den builds it: recorded (architecture, diagram 4). Posts review: confirmed, 2026-10-08
Fixtures	Commission seeds its own rows: "Untitled", "Old sketch" and the Workflow "Queue".	Recorded (question sheet, section 10)
When `is_archived` gets fixed	The tech lead's answer sends the bugs found outside the slice to their teams "after the slice". But the commission arms (WP-18) filter archived commissions, so this one is fixed before WP-18 uses it, on its own bug branch to `main`. Nothing calls it today (checked 2026-10-08), so the fix changes no behaviour.	Confirmed, 2026-10-08
Four entries in a commission	Your scope ruling gave four (question P2); `participants` arrives with its own slice. The approved fixture diagram and the corpus (My Den rulings, decision 9) show five; on this point the diagram is superseded for 0.1.	Recorded (question P2; question sheet, section 10)
Integration branches	All My Den slices merge into `feature/my-den` as soon as each passes review; the shell has its own branch and reaches `main` first; small fixes go straight to `main` (Delivery Plan, "Merge plan").	Confirmed, 2026-10-08; the shell's order, 2026-10-09
New in version 3: the time limits and bug reports		
Where the time limits land	Gecko builds them (WP-41) at units 1 to 2. They go straight to `main` on their own branch after the shell, before the identity branch's restack, which absorbs them; `feature/my-den` takes them in its restack onto `main`. Heron reviews them (his note; session handling), with Crow's QA.	Confirmed, 2026-10-09
The session check's wait for your profile	`/me` waits only a short time for your profile, inside the 5 s session check, and otherwise answers with the bare DID (WP-49, a small slice on the identity branch).	Confirmed, 2026-10-09
A sign-in start that runs out	The login form says "Zurfur isn't answering right now. Please try again." instead of the error page (in WP-41).	Confirmed, 2026-10-09
Bug reports are shared ground	Under the tech lead, built by the people the proposal names: moderation's programmers for the backend (Mastiff the contract and the backend, Goose the operator	Confirmed, 2026-10-09

Call	What and why	Status
	command; QA Lapwing), Finch for the web (QA Crow), Fennec for the corpus step and the merge. The builders are Kestrel's picks, confirmed by the tech lead for the leads to adjust.	
The bug reports' security reviewer	Mantis, the plugin team's QA, in the designer lane, so Heron's queue doesn't grow. Without a second reviewer, bug reports would move to 0.2 (the proposal's rule). Mantis runs at xhigh, following your lane rule for every security review; the tech lead corrected his earlier "high" on 2026-10-10 (`v2-carryover.md` , row 93).	Confirmed, 2026-10-09; effort xhigh, 2026-10-10
Bug reports in the merge order	The shell, the identity branch, bug reports, then My Den. `feature/bug-reports` lands after the identity branch and before My Den's one planned restack, which absorbs it; past unit 9 it waits for My Den and is 0.2's.	Proposed by the web lead; follows your cut line (D-34)

8 One-way doors in 0.1

Your rule of 2026-10-08 names four kinds of one-way door: a security boundary, the shape of stored data, a permission, and a published contract. **Your ruling of 2026-10-09 (D-36) narrows the second in pre-alpha:** a migration can be edited and every database rebuilt, so a migration is not a door, unless it deletes your real data; the `just migrate-add` rule stays. These are 0.1's candidates. Kestrel drafted the classification; the tech lead confirmed it on 2026-10-08 and found no other candidate; version 3 applies D-36. Of the three rows new in version 3, your ruling on `v2-carryover.md` , row 84, settles how the bug-report contract is reviewed; the table row follows D-36, and the gate's row rests on D-35, your own ruling on exactly that behaviour (its classification as a tightening is Kestrel's, not yet confirmed by the tech lead).

Candidate	Kind	How it is reviewed
den.proto, with the id-rule change (D-6 to D-10, D-14)	Published contract	**A door, approved:** you approved revision 2.1 (WP-25) as a whole on 2026-10-09 (D-6), with D-7 to D-10 and D-14 as ruled. Its text still comes to you on WP-13's PR (D-19), and nothing that depends on it starts before that PR merges.
Two migrations on the identity branch: a nullable hash column on `auth_request` , and an index on OAuth sessions	Stored data	**Not a door in pre-alpha (D-36).** You review them as part of #240 and #242, like any slice; WP-1 still checks that the index's version came from `just migrate-add` .
Deleting your stored sign-in sessions once (D-32)	Stored data (a one-time delete)	**A door, the exception D-36 keeps:** it deletes your real data, irreversibly on your machine, though nothing reads the sessions back and it removes a grant nothing in 0.1 uses. You review it on the scope slice's PR before it merges; Heron reviews the slice too. Confirmed by the tech lead, 2026-10-09, and by your ruling on D-32.
The dev profile's fixed-identity sign-in, reaching `main` with My Den	Security boundary	Ruled in by you (question P7), behind a Cargo feature and a refusal to start outside `dev` . Heron reviews it (WP-21) and again at the end of the slice (WP-22); you see it when `feature/my-den` merges.
A new callback code, `browser_mismatch` (D-5)	Contract (the login page's `?error=` codes)	Additive: an older page shows the generic message. You ruled D-5 (a).

Candidate	Kind	How it is reviewed
Refusing a PDS address that isn't a bare origin (D-11)	Security boundary (a refusal)	Tightening only, with no security effect. You ruled D-11 (a); Heron reviews the slice.
Retiring the owner's commission list (Product Design Document, D-28)	Contract (a removal)	Not a door in pre-alpha: nothing calls it, and a list can come back as an addition. It goes in as its own declared baseline-reset PR, which you review. Confirmed by the tech lead, 2026-10-09.
New in version 3: the `bug_report` table (D-37), and the participant table's by-user index (WP-17)	Stored data	**Not doors in pre-alpha (D-36).** D-37 is an ordinary decision, which you ruled (a); the proposal had the table's shape as a door before your ruling on migrations.
New in version 3: `bug_report.proto` (D-34)	Published contract	**A door at the alpha,** like den.proto: its spellings can't change once the alpha ships. It is additive (a new file and service), and its shape is the proposal's, which you ruled to follow (D-34). **You review the file itself on its PR** (WP-42) before it merges (your ruling on `v2-carryover.md`, row 84), and nothing builds on it before then.
New in version 3: the gate no longer reads an unreachable backend as "signed out" (D-35)	Security boundary (the sign-in gate)	A tightening: a `401` still signs you out; a network failure now shows the gate's own page, with no load or action behind it. Not a door; Heron reviews the slice (WP-41).

9 Technical risks

Each risk with its mitigation. Likelihood, impact and what to cut first are in the Delivery Plan's risk register.

Risk	Mitigation
Atta's read side is subtle: standing along the real path, mounts that re-anchor, filtering before the page.	A test per security check in My Den's security table; the recorded fallback (each node answers alone) if standing along the real path proves unworkable.
The host seam spans three teams, so the arms may not fit what Atta asks.	The host trait lands first; teams build against it; QA reviews after the interfaces settle (a lesson of the My Den round).
The one not-found drifts: a team maps its own error, or a route uses an extractor that answers differently.	One `NotFound` variant; no per-team mapping; tests that compare the bytes of every not-found case.
The PostgreSQL path is served for the first time: the reads, real roots, real names.	Den's PostgreSQL test setup; the den route suite on both stores; identity's and commission's seeders and cases on it; the timing test; Heron's reviews of each package and of the path as served.
Den's access check is one path that three teams' reads share, so a mistake in it leaks through every caller.	It is Atta's own resolve, not a second copy of the rule. Each caller gets a test that a node the Den refuses gets that read's usual answer. Heron reviews it with the arms.
Your root's name comes from your PDS on every Den answer, and the web makes up to 17 Den reads per page: a cold cache or an outage could stall My Den.	Identity bounds the lookup, runs one per DID at a time, and falls back to your DID, with a test whose PDS never answers. The web caps a page at 4 reads in flight.
`main`'s profile read lacks the resolver's guards (bare reqwest, a Bluesky fallback, the DID document's id unchecked).	Your real root builds against the profile ports, which `main` and the identity branch share, and is tested with a fake PDS; sign-in's merge (WP-5) replaces `main`'s profile read for `/me` and your root alike, and your root is checked live only after `feature/my-den` restacks onto it (the tech lead's call).

Risk	Mitigation
Both stacks serve My Den on 127.0.0.1, and cookies don't isolate by port (RFC 6265, section 8.5): if the dev profile keeps the PostgreSQL session store, each stack accepts the other's sessions.	The dev profile uses the in-memory session store (proposed by the identity lead; the tech lead's call, 2026-10-09); WP-21 tests both directions. Locally `.env` sets `ZURFUR_ENV=dev` for every recipe, so the Cargo feature alone keeps the dev profile off `just dev`; the `just dev` walk checks that the switch's page and its route are the ordinary not-found.
Den's PostgreSQL test setup is built before `feature/my-den` takes the hardened sign-in, so the restack may change the sign-in seam it drives.	It drives only the sign-in port; a small fix at the restack is accepted (the den lead).
The Accounts page and the tree could disagree about an Account on real data: the cost you accepted with D-2.	Both hide a deleted Account by default, and the Accounts list moves onto den's check.
Long-lived branches drift from `main`; root `Cargo.toml` and `Cargo.lock` collide.	Merge #236 and the identity branch early; you drive restacks (git-town); slices stay small.
A jacquard upgrade reopens a door around the guarded client.	The network-door guard test fails the build on the named doors; review check 19 re-reads jacquard at each upgrade.
New in version 3. The 5 s session check runs out while the API is up, because `/me` waits on your PDS for your profile.	`/me` bounds its profile wait inside the 5 s and falls back to the bare DID (WP-49, the tech lead's call); a test whose profile source never answers pins it. Until WP-49 reaches `main` with the identity branch, `main`'s `/me` reads the profile with no bound of its own, so a slow PDS shows the gate's page there rather than hanging.
A form post or a script navigation meets the gate's page: SvelteKit's client renders a form post's failure as the frame's error view (status 500), and a navigation reaches the page only after up to three 5 s session checks.	No action runs either way. WP-41 decides how a form post meets the gate and words it, and tests both; the bug-report form keeps its text. SC-9 bounds each request, not the navigation's retries.
A bug report's text aims instructions at an agent that reads it, or rewrites the operator's terminal.	In 0.1 every report is yours, because the form action refuses a post from anywhere but Zurfur's own pages (BR-10). The command fences, indents and escapes each report (BR-4), and a report's text is data, never instructions; who reads reports once others can send them is parked.
`feature/bug-reports` and `feature/my-den` edit the same files: the cookie surface in `routes/mod.rs`, the test that matches routes to the contract, the generated contract code, `zurfur-api.ts` and its mock, and the top bar, drawer and error view.	Bug reports land before My Den's one planned restack, which absorbs them; past unit 9 they wait for My Den and are 0.2's (the Delivery Plan, risk R19).

10 Parked: not in 0.1

Item	Why it waits	When it comes back
Rate limits on anonymous sign-in	Your ruling: they come with the User Page and its cache. The one limit 0.1 adds is the bug report's own, 10 an hour per reporter (Feature 4).	The User Page.
The record writer's transport (`AtprotoPublicRecords`)	No production caller yet.	The Hosted Repos decision, or its first caller.
The deployment checklist	Deployment is out of 0.1. Moderation's floor joins it (Product Design Document, D-26).	The first build that anyone but you can reach.
Character storage in PostgreSQL, and a way to create a character (the Product Design Document's D-1 (c))	Your ruling on D-1 is (b): (c) was 4 to 5 more units for a folder nothing in 0.1 can fill. A Character minted with	The Character slice, which must mint with the owner-first list.

Item	Why it waits	When it comes back
	today's minter gets the old custodial key list (a cold key, then the operational key), not the owner-first `[owner, pds, zurfur]` that Characters on ATProto rules (decisions/54427650, amendment of 2026-09-12).	
A route that admits a second Participant	Invitations can be issued and revoked, never accepted, so on PostgreSQL the creator is a commission's only Participant; commission's test adds bob by a test-only insert.	The slice that accepts invitations.
Creating a Workflow	No route creates one; your real `workflows` folders are empty (D-30).	The Workflow slice.
A post's key	`/post/<did>/<record key>` leaves out the collection, and atproto lets one record key repeat across collections: a one-way door at the first stored post (the posts study).	The first slice that stores a post.
`Account.id` drift in `account.proto`	A separate identity cleanup after the slice (the tech lead's answer on the question sheet).	After 0.1.
Sign-out leaving OAuth sessions stored	Found in triage; older than these branches.	Before PDS writes use the sessions.
Profile avatars load from the user's own PDS	Existing behaviour; the viewer's address reaches that host. The resolver design lists it as out of its scope.	When public profiles show other people's avatars (the User Page).
The type catalog	Type names are provisional (your ruling, question P10).	Before the alpha, or the names ship as they are.
The PostgreSQL keeper read's empty answer	An exemption from "no stubs" while PostgreSQL can't store characters (see "Calls the tech lead makes").	The Character slice, which replaces it as one of its acceptance criteria.
`participants`, view grants, bringing a commission into an Account, opening links, dotfiles, the maturity gate in your Den	Not in the first slice (your scope ruling, question P2).	Later slices.
New in version 3. The loading states: skeleton rows, "Still loading…", a quiet progress bar, and the tree's "Couldn't load. Retry." row	Your ruling on D-35: they wait. Planned ahead in `objectives/web.md`; their design-language items (Skeleton variants, the progress bar, InlineMessage's "still loading" and "isn't answering", and the tokens `--color-skeleton`, `--duration-skeleton`, `--delay-busy` and `--delay-slow`) go to the Claude Design brief. 0.1 keeps the layout ready for them (`objectives/web.md`, "Kept compatible in 0.1").	When you place them, after the Claude Design sync.
Signed-out bug reporting, with its anti-abuse	In 0.1 only a signed-in user reports (D-34); a signed-out write has nothing to limit it by but an IP address.	The alpha's invite slice.
Who reads bug reports once others can send them, and how agents treat their text	In 0.1 every report is yours (BR-10).	The same slice.
A "Your reports" page, with reports moving into the reporter's Den	No second reporter exists yet.	When one does.

Item	Why it waits	When it comes back
The operator's read over HTTP behind an operator's sign-in; closing a report and telling the reporter	The read runs in-process in 0.1, like `migrate`; your command-line ruling keeps user commands on HTTP because an in-process tool holds database admin power.	With moderation's console and moderator sign-in, from the alpha.
Screenshots in bug reports	An image is an upload, behind moderation's scan gate (Product Design Document, D-26).	After the scan gate.
Forwarding bug reports to a tracker or a mailbox	The repository is public, and no mailer exists; whatever forwards a report must drop the page address or ask the reporter.	When you ask for it, with its own security review.
Bug reports' retention, and what an account's deletion does to them	Only your reports exist in pre-alpha.	Before the alpha, with moderation's retention rule.
A pointer from the bug form to the content report	Moderation's report path doesn't exist yet.	When that path ships.
New in version 4. A Finder-style column view: each opened folder becomes a new column	Your note on D-3 (Product Design Document): "I enjoy the way MacOS does it most with the columns as you go opening. This last thing may be deferred as a theme in the future." 0.1's tree behaves like a file explorer, as you ruled.	As a future theme or view, when you place it.
New in version 4. A link (symlink) to an archived node	Your ruling on D-15 leaves it open; 0.1 has no symlink.	The first slice with a symlink.
New in version 4. The payload hash, `Payload` and attributes in Atta's metadata	No payload in 0.1, and no code path serves one (your rulings on `v2-carryover.md`, rows 9 and 12).	The first payload.

11 What changed in version 4

Version 4 goes with the Product Design Document's version 5 and the Delivery Plan's version 4. It folds in your rulings of 2026-10-09 on every decision here and on the carry-over's questions (`rulings-2026-10-09.md`), and the tech lead's answer of 2026-10-10 on Mantis's effort. No decision is renumbered, and no package or unit changes; in the Delivery Plan your review of den.proto's text moves inside WP-13, due by unit 5, and one slack figure is corrected (R4).

Change	Driven by
Decisions: D-6 to D-16 and D-37 are decided blocks, "Ruled by the Engineer, 2026-10-09", each as Claude answered it, with Claude's proposal kept above the ruling. "Decisions needed: technical" becomes "Decisions: technical", and its opening says what you ruled. The summary counts 13, all ruled.	your rulings
den.proto is approved as a whole (revision 2.1, D-6): the summary, the overview's dashed line, "Size and checks" and the door table say so. Its text still comes to you on WP-13's PR, with D-19's line in the lifecycle file.	your rulings
D-15 is approved with its redraw. Its inner question is ruled: a typed path to an archived node opens only through the viewer's own relationship to it, never its public level alone; the other, which reach a link to one uses, stays open until the first slice with a symlink. The rule table's note cites the ruling, and the den lead corrects the amendment's own table before WP-12 builds the host trait (was: before the review sitting). The joined component diagram is approved (Feature 3; the calls).	your rulings
No code path serves a payload (carry-over row 12): Feature 3's "Data", SEC-4, and the end-of-slice review's reading of the question sheet's finding 19.	your approval

Change	Driven by
Atta's read side builds on your draft (carry-over row 9): the summary and Feature 3's architecture list the changes the rulings force; the payload hash, `Payload` and attributes wait for the first payload.	your ruling
`bug_report.proto` is a door you review on its PR (carry-over row 84): Feature 4's contract, the summary and the door table. Mantis reviews at xhigh (Feature 4's opening; the reviewer row).	your ruling; the tech lead
The names row allows the two joiners and nothing else from Cf (D-29). D-5, D-7, D-11, D-16, D-19 and D-37 are cited as ruled wherever they read "Claude's answer".	your rulings
Parked: the column view (your note on D-3), a link to an archived node (D-15), and the payload hash, `Payload` and attributes.	your rulings and note
D-13 points to the tech lead's offer of one design-sync for every corpus follow-up your rulings create (`v2-carryover.md` , row 94).	the rulings file
The door table: den.proto's row is approved, D-5's, D-11's and D-37's rows cite your rulings, and the opening says how your ruling on carry-over row 84 settles the bug-report contract's review.	your rulings
After Owl's QA (`qa.md` , "QA challenges, version 5"): D-6 says the PR's den.proto is revision 2.1 unchanged, and lists what the PR adds around it; the decisions' opening says you could review the text, not that you did; the names row refuses a name of joiners alone (D-29); D-15's link question is worded as deferred, in the ruling, the caption and the summary; D-36's exception is named as yours; D-11's reason moves out of the ruling box; D-37 labels Kestrel's reading of D-36; the door table's gate row rests on D-35, and the session delete's row cites D-32.	den QA

12 What changed in version 3

Version 3 goes with the Product Design Document's version 4. It follows your four rulings of 2026-10-09 (D-1 is (b); D-34, bug reports; D-35, the time limits; D-36, migrations in pre-alpha), the web lead's bug-report proposal with its QA resolutions, and the tech lead's calls of the same day. No decision is renumbered; D-36 and D-37 are new.

Change	Driven by
Summary: the side feature and the time limits; where bug reports stand; the `bug_report` table; D-36 on migrations; the two doors reviewed on their PRs; 13 decisions.	your rulings
Architecture overview: a note on what isn't drawn (the time limits inside the web box; bug reports beside the path).	Kestrel
Feature 2: a new section, "Time limits and the 'isn't answering' page": the limits in one fetch helper, the 15 s request deadline, the gate's own page and its headers, what a timeout shows elsewhere, a sign-in start's 10 s, the session check's bounded wait for your profile (WP-49), the tests, and a sequence diagram.	your ruling; web; Crow's QA; the tech lead
Feature 3: "Claude's answer to D-1" reads "your ruling on D-1" (the data section and the parked character-storage row).	your ruling
New Feature 4, bug reports: architecture, `bug_report.proto` , the `bug_report` table, the security table (BR-1 to BR-9) with the two gaps that predate it, and the proposal's diagram A as a sequence.	your ruling; the proposal
Decisions: D-36, your ruling that a migration isn't a door in pre-alpha unless it deletes your real data; D-37, the stored bug report's shape, on Claude's answer.	your ruling; identity; the proposal
"Calls the tech lead makes": six new rows (where the time limits land, the session check's wait, a sign-in start that runs out, bug reports as shared ground, Mantis as their reviewer, their place in the merge order).	the tech lead; web
"One-way doors in 0.1": rewritten under D-36. The two identity migrations, the participant index and the `bug_report` table are not doors; the session delete stays one; `bug_report.proto` is a door reviewed on its PR; the gate's change is a tightening.	your ruling; Kestrel
Technical risks: four new (the session check against your PDS, the gate's page on a form post or a script navigation, a report's text aimed at an agent or a terminal, the	web; the proposal; the tech lead

Change	Driven by
shared files with My Den). Parked: the loading states and the proposal's eight parked items; the rate-limit row names the bug report's limit.	
After den QA's pass (`qa.md` , "QA challenges, version 4"): Feature 2 states what the installed SvelteKit does with the gate's page (a form post becomes the frame's error view; a script navigation retries before a full load), escapes "Try again", lets WP-49's profile fetch finish so the cache fills, words a timed-out write, and drops TextInput from the built components; Feature 4 gains BR-10 (the form action refuses a post from anywhere but Zurfur's own pages, in development too), the `Referer` rule, CR LF turned into LF, the phone's top-bar icon, which caps count bytes, and the dev profile's store in WP-50; D-36 carries Kestrel's reading of your exception (a migration that has run on your own database is changed by a new one); D-37's latest adjustment is before bug reports reach `main` ; the three new door rows and Mantis's effort are marked for the tech lead to confirm.	den QA

13 What changed in version 2

Version 2 goes with the Product Design Document's version 3. It follows your ruling that 0.1 joins the halves (D-1), the four leads' joining pages (`objectives/join-*.md`) and the tech lead's calls of 2026-10-09. Decision numbers are unchanged.

Change	Driven by
Header: the stage in the subtitle. Summary: the shell is built; four follow-up slices, the scope slice among them; My Den served from PostgreSQL after a real sign-in; one door carrying six decisions.	your ruling; web; plugin's earlier correction
Architecture overview: the den routes on every stack, den's access check, `adapter-pg` served, and your root's name from your PDS.	den, identity
Feature 1: the identity-only scope in the security table, and the one-time delete of the old sessions (D-17, D-32).	identity
Feature 2: one frame on every stack, slice 1b's switch deleted; the cap of 4 parallel Den reads; generic page titles.	web
Feature 3, architecture: two stacks, one code path; den's access check; your real root; who rewires which read.	den, identity, commission
Feature 3, contract: the routes served on every stack, the request-field sentence, the route pin, and the retired owner's list (D-28).	den, commission; the earlier carry-over
Feature 3, data: the keeper read confirmed and live on `just dev` ; PostgreSQL served; den's PostgreSQL test setup; your own data from the fill script; legacy names without `just db-reset` ; fixtures all Private, with "Old sketch" and "Queue".	den, identity, commission; the tech lead
Feature 3, security: SEC-6 gated on equal database statements; SEC-9's gate build; new SEC-10 (den's access check), SEC-11 (your root's name) and SEC-12 (no other User named); the names row; the end-of-slice review's wider list; the two access paths converging in 0.1; whole-folder reads accepted, as proposed.	den, identity, web, commission; the tech lead
D-15: the rule table, corrected to match the revised answer on view grants, replacing the amendment's own, and flagged for den QA.	the tech lead's call on the rule table
"Calls the tech lead makes": the calls of 2026-10-09, including six more after den QA (the session store, keeping the dev profile, whole-folder reads, the joined diagram's approval, the review's size, QA of the joining pages); "Keep My Den off the everyday stack" replaced.	the tech lead; den QA
Feature 3: a new figure, the changelog read behind den's check, awaiting your approval with D-15; the read cap's effect on the pool marked unverified; the two access paths' agreement explained by what 0.1 mounts. After the team QA passes: stored names load as text and are checked only at write; SEC-10 names each read's refused answer and the pairs of causes, and asks for equal statements on a real path before the arms start; SEC-11 records its lookups; SEC-9 has two locks; the session delete is a migration; the merge order names ten shared web files.	den, web, identity and commission QA; the tech lead

Change	Driven by
One-way doors: two new candidates, confirmed by the tech lead: the retired list (not a door) and the one-time session delete (a door, which you review on its PR).	Kestrel; the tech lead
Technical risks: the PostgreSQL path served, the shared check, the root-name lookup, `main`'s unguarded profile read, two stacks on one cookie, the test setup's restack, the Accounts page against the tree.	den, identity, web
Parked: the follow-up and the timing re-test leave (now in 0.1); character storage and its key list, a second Participant, Workflow creation and a post's key arrive; the deployment checklist comes back at the first reachable build.	den, identity, commission, posts, moderation